

TRƯỜNG ĐẠI HỌC THỦY LỢI
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO TỔNG KẾT
ĐỀ TÀI: MÔ HÌNH PHÂN LỚP DỮ LIỆU KHÔNG CHẮC CHẮN

Giảng viên hướng dẫn: **TS. Nguyễn Quỳnh Diệp**

Khoa: **Công nghệ thông tin**

Nhóm sinh viên thực hiện:

Nguyễn Thế Toàn	64CNTT1	2251061894
Lê Thanh Bình	64CNTT2	2251061725
Nguyễn Thị Thanh Vân	64CNTT1	2251061922
Lê Hà Phương	64CNTT1	2251061863
Lưu Hữu Khánh	64CNTT1	2251061807

Hà Nội, 2025

MỤC LỤC

MỞ ĐẦU	3
CHƯƠNG 1: CƠ SỞ LÝ THUYẾT	4
1.1. Tổng quan dữ liệu không chắc chắn	4
1.3. Một số cách biểu diễn dữ liệu không chắc chắn	5
1.4. Một số phương pháp phân lớp không chắc chắn	9
CHƯƠNG 2: XÂY DỰNG MÔ HÌNH PHÂN LỚP DỮ LIỆU KHÔNG CHẮC CHẮN	11
2.1. Mô hình phân lớp UK-medoids	11
2.1.1. Mô hình hóa dữ liệu không chắc chắn UK-medoids	11
2.1.2. Khoảng cách không chắc chắn UK-medoids	12
2.1.3. Thuật toán UK-medoids	14
2.2. Mô hình phân lớp UK-means	15
2.2.1. Bài toán phân lớp không chắc chắn UK-means	15
2.2.2. Thuật toán UK-means	16
CHƯƠNG 3: CÀI ĐẶT THỰC NGHIỆM	18
3.1. Cài đặt thuật toán UK-medoids	18
3.1.1. Bộ dữ liệu	18
3.1.2. Xây dựng mô hình	19
3.1.3. Đánh giá	39
3.2. Cài đặt thuật toán UK-means	41
3.2.1. Bộ dữ liệu	41
3.2.2. Xây dựng mô hình	43
3.3.3. Đánh giá	51
3.3. So sánh, đánh giá các thuật toán	53
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	58
TÀI LIỆU THAM KHẢO	59

DANH MỤC BẢNG BIỂU

Bảng 1.1: Tổng hợp các phương pháp biểu diễn dữ liệu không chắc chắn	11
Bảng 3.1. Mô tả dữ liệu thu được sau khi tiền xử lý	21
Bảng 3.2. Bảng so sánh F-Measure giữa UK-Medoids và K-Medoids.....	42
Bảng 3.3. Mô tả dữ liệu đầu vào cho thuật toán UK-means sau tiền xử lý.....	44
Bảng 3.4. Bảng so sánh F-Measure giữaUK-means, UK-Medoids và K-Medoids	53
Bảng 3.5. Bảng kết quả so sánh các thuật toán và phần mềm ELKI.....	59

DANH MỤC HÌNH ẢNH

Hình 3.1. Minh họa phân lớp tập Iris với phân phối xác suất Uniform	42
Hình 3.2. Minh họa phân lớp tập Iris với phân phối xác suất Normal	43
Hình 3.3. Minh họa phân lớp tập Iris với phân phối xác suất Binomial	43
Hình 3.4. Minh họa phân lớp tập Iris với phân phối xác suất Uniform	54
Hình 3.5. Minh họa phân lớp tập Iris với phân phối xác suất Normal	54
Hình 3.6. Minh họa phân lớp tập Iris với phân phối xác suất Binomial	55

MỞ ĐẦU

Trong bối cảnh bùng nổ của dữ liệu lớn, việc xử lý và phân tích dữ liệu không chắc chắn trở thành một thách thức ngày càng lớn. Dữ liệu không chắc chắn xuất hiện rộng rãi trong nhiều lĩnh vực, từ khoa học tự nhiên đến khoa học xã hội, và đặc biệt phổ biến trong các ứng dụng liên quan đến cảm biến, định vị, và mô hình hóa hệ thống phức tạp. Việc áp dụng các thuật toán phân lớp truyền thống, vốn được thiết kế cho dữ liệu xác định, có thể dẫn đến kết quả không chính xác và thiếu tin cậy khi đối mặt với dữ liệu không chắc chắn.

Đề tài nghiên cứu này tập trung vào việc khám phá và phát triển các phương pháp phân lớp dữ liệu không chắc chắn, với mục tiêu cải thiện độ chính xác và hiệu quả của quá trình phân tích dữ liệu trong các tình huống thực tế. Chúng tôi đặc biệt quan tâm đến việc áp dụng và mở rộng các thuật toán UK-means và UK-medoids, hai phương pháp phân lớp hứa hẹn trong việc xử lý dữ liệu không chắc chắn. Ngoài ra, chúng tôi cũng đánh giá sự hữu ích của thư viện ELKI, một công cụ mạnh mẽ trong lĩnh vực khai phá dữ liệu, trong việc triển khai và so sánh các thuật toán phân lớp.

Nghiên cứu này không chỉ mang tính lý thuyết mà còn có giá trị ứng dụng cao. Các kết quả và phương pháp được đề xuất có thể được áp dụng trong nhiều lĩnh vực, từ phân tích dữ liệu cảm biến trong môi trường thông minh đến phân lớp dữ liệu vị trí trong các ứng dụng di động. Chúng tôi hy vọng rằng nghiên cứu này sẽ đóng góp vào sự phát triển của các phương pháp phân tích dữ liệu không chắc chắn, giúp chúng ta hiểu rõ hơn về thế giới phức tạp xung quanh.

CHƯƠNG 1: CƠ SỞ LÝ THUYẾT

1.1. Tổng quan dữ liệu không chắc chắn

Trong những năm gần đây, việc quản lý dữ liệu không chắc chắn ngày càng trở nên quan trọng hơn, đặc biệt là do sự gia tăng của các ứng dụng phân cứng thu thập dữ liệu gần đúng. Chẳng hạn, các cảm biến thường gặp phải nhiễu đáng kể trong quá trình đo đạc do sự không chính xác khi truy xuất, truyền tải dữ liệu hoặc do mất điện. Trong nhiều trường hợp, hệ thống có thể cung cấp thông tin về sai số ước lượng trong luồng dữ liệu. Đây là thông tin rất hữu ích cho quá trình khai phá dữ liệu, vì nó có thể được tận dụng để nâng cao chất lượng của các kết quả thu được.

Trong khoa học máy tính, dữ liệu không chắc chắn là dữ liệu có chứa nhiều khiến dữ liệu lệch khỏi giá trị chính xác, dự định hoặc ban đầu. Trong thời đại dữ liệu lớn, sự không chắc chắn hoặc tính xác thực của dữ liệu là một trong những đặc điểm xác định của dữ liệu. Dữ liệu liên tục tăng về khối lượng, tính đa dạng, tốc độ và tính không chắc chắn (1/tính xác thực). Ngày nay, dữ liệu không chắc chắn có rất nhiều trên web, trong các mạng cảm biến, trong các doanh nghiệp ở cả nguồn có cấu trúc và không có cấu trúc. Ví dụ, có thể có sự không chắc chắn liên quan đến địa chỉ của khách hàng trong tập dữ liệu doanh nghiệp hoặc các chỉ số nhiệt độ do cảm biến thu được do cảm biến bị lão hóa. Năm 2012, IBM đã chỉ ra việc quản lý dữ liệu không chắc chắn ở quy mô lớn trong báo cáo triển vọng công nghệ toàn cầu của mình. IBM cũng đưa ra một phân tích toàn diện hướng đến tương lai từ ba đến mười năm nhằm xác định các công nghệ đột phá quan trọng sẽ thay đổi thế giới. Để đưa ra quyết định kinh doanh tự tin dựa trên dữ liệu thực tế, các phân tích nhất thiết phải tính đến nhiều loại sự không chắc chắn khác nhau có trong lượng dữ liệu rất lớn. Phân tích dựa trên dữ liệu không chắc chắn sẽ ảnh hưởng đến chất lượng của các quyết định tiếp theo, do đó, mức độ và loại không chính xác trong dữ liệu không chắc chắn này không thể bị bỏ qua.

Dữ liệu không chắc chắn (uncertain data) là khái niệm chỉ các loại dữ liệu không chính xác tuyệt đối, có thể chứa sai số, thiếu hụt, hoặc không rõ ràng do nhiều nguyên nhân khác nhau như: lỗi đo lường, ước lượng, cảm biến không chính xác, hoặc sự không đầy đủ của thông tin.

1.2. Nguyên nhân gây ra tính không chắc chắn trong dữ liệu

Dữ liệu không chắc chắn được tìm thấy trong lĩnh vực mạng cảm biến (IoT, robot, xe tự hành,...); văn bản nơi văn bản nhiễu được tìm thấy rất nhiều trên phương tiện truyền thông xã hội, web và trong các doanh nghiệp nơi dữ liệu có cấu trúc và không có cấu trúc có thể cũ, lỗi thời hoặc hoàn toàn không chính xác (xử lý ngôn ngữ tự nhiên); dữ liệu y tế như chuẩn đoán bệnh có xác suất; trong mô hình hóa nơi mô hình toán học chỉ có thể là một phép xấp xỉ của quy trình thực tế. Khi biểu diễn dữ liệu như vậy trong cơ sở dữ liệu, cần phải chọn một mô hình cơ sở dữ liệu không chắc chắn phù hợp. Tính không chắc chắn trong dữ liệu có thể bắt nguồn từ nhiều nguyên nhân khác nhau, bao gồm:

- Sai số đo lường: Sai số phát sinh từ thiết bị đo hoặc kỹ thuật thu thập dữ liệu không chính xác. Chẳng hạn, một cảm biến nhiệt độ có thể ghi nhận nhiệt độ phòng là $25^{\circ}\text{C} \pm 1^{\circ}\text{C}$ do giới hạn độ chính xác của thiết bị.
- Yếu tố môi trường: Các yếu tố ngẫu nhiên hoặc không thể kiểm soát trong môi trường, như sự biến đổi của thời tiết hoặc điều kiện tự nhiên. Ví dụ, dự báo thời tiết thường không thể đạt độ chính xác tuyệt đối do ảnh hưởng từ nhiều yếu tố môi trường phức tạp.
- Lỗi do con người: Sự mơ hồ trong ngôn ngữ, lỗi trong quá trình nhập liệu hoặc thiếu thông tin trong khâu thu thập dữ liệu. Ví dụ, một nhân viên nhập nhầm chiều cao của một người từ 175 cm thành 170 cm có thể dẫn đến sai lệch.
- Mô hình hóa: Tính không chắc chắn phát sinh từ việc sử dụng các giả định hoặc mô hình không phản ánh đầy đủ thực tế. Ví dụ, mô hình dự đoán kinh doanh có thể không lường trước được những sự kiện đột biến như đại dịch hay suy thoái kinh tế.

1.3. Một số cách biểu diễn dữ liệu không chắc chắn

a) Khoảng giá trị (Interval Data): Dữ liệu không có giá trị cụ thể, chỉ biết nằm trong một khoảng nhất định.

Ví dụ: Dữ liệu từ cảm biến nhiệt độ

Thời gian	Nhiệt độ ($^{\circ}\text{C}$)	Sai số ($\pm$)
2025-04-18 10:00	30.2	± 0.5
2025-04-18 10:10	29.8	± 0.7
2025-04-18 10:20	30.5	± 0.4

Giá trị đo được có thể bị ảnh hưởng bởi các yếu tố như điều kiện môi trường, độ nhạy của thiết bị hoặc lỗi kỹ thuật. Vì vậy, mỗi giá trị đo không hoàn toàn chính xác tuyệt đối, mà thường nằm trong một khoảng tin cậy nhất định, chẳng hạn: 30.2 ± 0.5 .

b) Phân phối xác suất (Probabilistic Data):

Sử dụng phân phối xác suất để mô tả toàn bộ dải giá trị mà dữ liệu có thể nhận, cùng với xác suất xảy ra của từng giá trị. Cách tiếp cận này mang lại nhiều lợi ích, bao gồm:

- Thể hiện tính chắc chắn trong dữ liệu một cách rõ ràng và có phương pháp.
- Hỗ trợ phân tích, suy luận và dự đoán với mức độ tin cậy cao hơn.
- Là cơ sở cho các mô hình học máy theo hướng xác suất, chẳng hạn như mô hình Bayesian và phân tích rủi ro trong các hệ thống phức tạp.

Ví dụ: Thay vì ghi nhận một nhiệt độ đo được là 30.2°C , ta có thể biểu diễn nó bằng một phân phối chuẩn (Normal Distribution) với trung bình 30.2 và độ lệch chuẩn 0.5 : $T \sim N(30.2, 0.5^2)$. Điều này có nghĩa:

- Giá trị trung bình (mean) là 30.2°C
- Độ lệch chuẩn (standard deviation) là 0.5°C
- Nhiệt độ hôm nay 30.7°C với xác suất 68%, khoảng 68% nhiệt độ thực nằm trong khoảng $[29.7^{\circ}\text{C}, 30.7^{\circ}\text{C}]$

c) Tập hợp rời rạc có trọng số (Discrete Values with Probabilities):

Trong một số tình huống, dữ liệu không được xác định dưới dạng một giá trị duy nhất mà thay vào đó là một tập hợp các giá trị rời rạc, mỗi giá trị đi kèm với một trọng số xác suất thể hiện mức độ tin cậy. Một điểm dữ liệu có thể nhận nhiều giá trị khác nhau với xác suất tương ứng. Phương pháp này có ưu điểm:

- Cho phép biểu diễn rõ ràng các khả năng thay vì duy nhất một giá trị.
- Tăng tính linh hoạt và chính xác trong các bước xử lý tiếp theo (phân tích, suy luận, ra quyết định).

Cấu trúc biểu diễn:

$$\{(x_1, p_1), (x_2, p_2), \dots, (x_n, p_n)\}$$

Trong đó:

- x_i : là giá trị rời rạc có thể có.

- p_i : là xác suất (trọng số) tương ứng với x_i sao cho

Ví dụ: Một cảm biến đo nhiệt độ và trả về kết quả không chắc chắn. Tập dữ liệu không chắc chắn được biểu diễn như sau:

$$\{(20^{\circ}\text{C}, 0.1), (23^{\circ}\text{C}, 0.6), (24^{\circ}\text{C}, 0.3)\}$$

d) Lý thuyết mờ:

Lý thuyết mờ (Fuzzy Logic) mô tả một giá trị không chắc chắn hoặc không rõ ràng bằng cách sử dụng tập mờ (fuzzy set). Thay vì nói một giá trị là đúng hoặc sai, dữ liệu mờ cho phép nó đúng một phần nào đó, thông qua một hàm thuộc (membership function) có giá trị trong khoảng $[0,1]$. Thay vì giá trị chính xác, dữ liệu được mô tả bằng các khái niệm mơ hồ như "cao", "thấp", "ấm", "lạnh".

Ví dụ: Một ngày được mô tả là "hơi lạnh" mà không có nhiệt độ cụ thể.

e) Lý thuyết Dempster-Shafer

Lý thuyết Dempster-Shafer (còn gọi là Lý thuyết bằng chứng) là một mô hình suy luận xác suất tổng quát cho phép xử lý không chắc chắn và sự thiếu thông tin (ignorance), tốt hơn so với xác suất truyền thống. Mục đích của lý thuyết Dempster-Shafer:

- Biểu diễn niềm tin vào các giả thuyết có thể xảy ra.
- Cho phép gán niềm tin không chỉ cho một giá trị đơn lẻ, mà cho tập hợp các giá trị.
- Phù hợp trong trường hợp không đủ thông tin để gán xác suất cụ thể.

Ví dụ: Nhận diện đối tượng qua cảm biến

Giả sử có một hệ thống giám sát dùng 2 cảm biến để xác định đối tượng phía trước là: $\Theta = \{\text{Người (H)}, \text{Vật nuôi (A)}, \text{Vật thể (O)}\}$

Thông tin từ cảm biến:

- Tin rằng là Người: 0.6
- Không phân biệt được là Người hay Vật nuôi: 0.3
- Không chắc chắn gì, nên gán cho toàn bộ Θ : 0.1

$$m_1(H)=0.6, m_1(\{H,A\})=0.3, m_1(\{H,A,O\})=0.1$$

f) Entropy

Entropy đo lường lượng thông tin trung bình hoặc mức độ không chắc chắn của một biến ngẫu nhiên. Càng không chắc chắn, entropy càng cao. Càng chắc chắn (ít lựa chọn), entropy càng thấp.

Cho một biến ngẫu nhiên X có các giá trị $x_1, x_2, \dots, x_n$ với xác suất tương ứng $p_1, p_2, \dots, p_n$

$$H(X) = - \sum_{i=1}^n p(x_i) \log_2 p(x_i)$$

Trong đó:

- $H(X)$: Entropy của biến X
- $p(x_i)$: Xác suất của kết quả x_i
- $\log_2$: Thông tin được đo bằng bit

Ví dụ: Biến X có 3 khả năng:

Giá trị x_i	Xác suất $p(x_i)$
Người (H)	0.5
Vật nuôi (A)	0.3
Vật thể (O)	0.2

Tính entropy: $H(X) = -(0.5 \times \log_2 0.5 + 0.3 \times \log_2 0.3 + 0.2 \times \log_2 0.2) = 1.4855$ bits

Vì Entropy ≈ 1.49 bits nên có mức độ không chắc chắn tương đối cao.

g) Lý thuyết tập thô

Lý thuyết tập thô (Rough Set) là một phương pháp nhằm biểu diễn và xử lý sự không chắc chắn và mơ hồ trong dữ liệu, dựa trên khái niệm quan hệ tương đương và xấp xỉ tập hợp. Khi không thể xác định một cách chính xác liệu một đối tượng có thuộc về một tập hợp nào đó hay không (do thiếu thông tin), Rough Set chia tập hợp thành hai phần:

- Xấp xỉ dưới (Lower approximation): gồm các đối tượng chắc chắn thuộc về khái niệm cần xét.
- Xấp xỉ trên (Upper approximation): gồm các đối tượng có khả năng thuộc về khái niệm (chưa thể khẳng định rõ ràng).

Phần nằm giữa hai xấp xỉ này được gọi là vùng biên (boundary region) - chính là khu vực thể hiện sự không chắc chắn trong việc phân loại đối tượng.

Phương pháp	Loại không chắc chắn	Ví dụ ứng dụng
Xác suất	Ngẫu nhiên	Phân loại, dự đoán
Dữ liệu mờ	Mơ hồ	Điều khiển mờ, NLP
Dempster-Shafer	Không chắc chắn, không biết	Hệ chuyên gia
Khoảng giá trị	Không rõ chính xác	Dữ liệu cảm biến, đo lường
Entropy	Đo mức độ bất định	Lựa chọn mô hình, thông tin
Rough Set	Không rõ ràng về ranh giới	Khai phá dữ liệu
Bayesian Inference	Cập nhật niềm tin	Học máy, thống kê Bayes
Monte Carlo	Mô phỏng bất định phức tạp	Khoa học tính toán, mô phỏng rủi ro

Bảng 1.1: Tổng hợp các phương pháp biểu diễn dữ liệu không chắc chắn

1.4. Một số phương pháp phân lớp không chắc chắn

Việc xử lý dữ liệu không chắc chắn đang ngày càng trở nên quan trọng trong nhiều lĩnh vực ứng dụng khác nhau. Thực tế, sự không chắc chắn trong dữ liệu có thể phát sinh một cách tự nhiên từ nhiều nguyên nhân, chẳng hạn như yếu tố ngẫu nhiên tiềm ẩn trong quá trình tạo hoặc thu thập dữ liệu, sai số trong các phép đo vật lý, hoặc do dữ liệu đã trở nên lỗi thời.

Tùy theo ngữ cảnh ứng dụng, có nhiều cách tiếp cận và định nghĩa khác nhau về sự không chắc chắn. Nhìn chung, sự không chắc chắn có thể được biểu diễn ở nhiều cấp độ khác nhau như bằng dữ liệu, từng bộ dữ liệu hoặc từng thuộc tính và thường được mô hình hóa thông qua các phương pháp như mô hình mờ, mô hình dựa trên bằng chứng, hoặc mô hình xác suất.

Ở dạng cấp độ thuộc tính, dữ liệu có chứa sự không chắc chắn được mô hình hóa thông qua các mô hình xác suất. Thay vì được biểu diễn dưới dạng vector xác định như truyền thống, dữ liệu không chắc chắn thường được mô tả bằng hàm mật độ xác suất (probability density functions-PDF), phản ánh khả năng dữ liệu đó xuất hiện tại từng vị trí trong không gian nhiều chiều.

Việc xử lý các dữ liệu không chắc chắn đã nảy sinh một số vấn đề trong quản lý dữ liệu và khai phá dữ liệu. Đặc biệt, việc tổ chức dữ liệu không chắc chắn là một thách thức vì sự khó khăn nội tại nằm dưới các khái niệm khác nhau về sự không chắc chắn. Là một nhiệm vụ khám phá chính của khai phá dữ liệu, phân lớp là việc tổ chức một tập hợp các điểm dữ liệu (phân loại chưa biết) thành các nhóm có ý nghĩa (lớp), dựa trên các mối quan hệ thú vị được phát hiện trong dữ

liệu. Các điểm dữ liệu trong một lớp sẽ rất giống nhau, nhưng sẽ rất khác so với các điểm dữ liệu trong các lớp khác. Một trong những phương pháp phân lớp phổ biến nhất được thể hiện bằng phân lớp phân vùng, trong đó gán lặp lại các điểm dữ liệu vào các lớp theo một hàm khoảng cách/tương đồng xác định. Một yếu tố quan trọng trong phân lớp phân vùng là cách đưa ra khái niệm về nguyên mẫu lớp. Cụ thể, một nguyên mẫu lớp có thể được định nghĩa là một tâm, là điểm dữ liệu “mean” trong lớp, hoặc là một medoid, là điểm dữ liệu thực tế gần nhất với tất cả các điểm dữ liệu khác trong lớp. Các thuật toán K-means và K-medoids là các phương pháp mẫu của phân lớp phân vùng dựa trên tâm và dựa trên medoid, tương ứng.

Thuật toán K-means đã được điều chỉnh cho miền dữ liệu không chắc chắn. Tuy nhiên, thuật toán kết quả, có tên là UK-means, có hai điểm yếu chính. Đầu tiên, các trọng tâm lớp được định nghĩa là các điểm dữ liệu xác định và được tính toán như giá trị trung bình của các giá trị kỳ vọng trên các PDF của các điểm dữ liệu không chắc chắn trong lớp; việc định nghĩa các trọng tâm theo cách này có thể dẫn đến mất độ chính xác, vì chỉ các giá trị kỳ vọng của các pdf của các điểm dữ liệu không chắc chắn mới được tính đến. Thứ hai, việc tính toán khoảng cách kỳ vọng (ED) giữa các trọng tâm lớp và các điểm dữ liệu không chắc chắn tốn kém về mặt tính toán, vì nó yêu cầu các ước tính tích phân số không tầm thường; điều này thể hiện một nút thắt về hiệu quả ở mỗi lần lặp lại của thuật toán.

UK-medoids là một thuật toán phân lớp các điểm dữ liệu không chắc chắn, được xây dựng dựa trên mô hình phân lớp K-medoids. Thuật toán này sử dụng hàm khoảng cách chuyên biệt dành cho dữ liệu không chắc chắn, không chỉ giới hạn ở việc sử dụng các giá trị vô hướng như kỳ vọng từ các hàm mật độ xác suất (PDF) liên quan đến các điểm dữ liệu. Nhờ vậy, nó cho phép ước lượng khoảng cách giữa hai điểm dữ liệu không chắc chắn một cách chính xác hơn, từ đó nâng cao đáng kể chất lượng phân lớp. Bên cạnh đó, UK-medoids không yêu cầu lặp lại các phép tính phức tạp tại mỗi vòng lặp. Trên thực tế, việc tính toán khoảng cách giữa các điểm dữ liệu không chắc chắn chỉ được thực hiện một lần, góp phần cải thiện hiệu suất rõ rệt so với thuật toán UK-means.

CHƯƠNG 2: XÂY DỰNG MÔ HÌNH PHÂN LỚP DỮ LIỆU KHÔNG CHẮC CHẮN

2.1. Mô hình phân lớp UK-medoids

2.1.1. Mô hình hóa dữ liệu không chắc chắn UK-medoids

Việc biểu diễn các điểm dữ liệu không chắc chắn ở cấp độ thuộc tính theo truyền thống được thực hiện bằng cách sử dụng hai mô hình, cụ thể là mô hình không chắc chắn đa biến và mô hình không chắc chắn đơn biến.

Sử dụng mô hình không chắc chắn đa biến, một điểm dữ liệu không chắc chắn m -chiều được định nghĩa theo vùng m -chiều và hàm mật độ xác suất đa biến, lưu trữ xác suất theo đó biểu diễn chính xác của điểm dữ liệu trùng với bất kỳ điểm nào trong vùng. Trong mô hình không chắc chắn đơn biến, điểm dữ liệu không chắc chắn m -chiều có, đối với mỗi thuộc tính, một khoảng và hàm mật độ xác suất đơn biến gán giá trị xác suất cho bất kỳ điểm nào trong khoảng.

Một điểm dữ liệu không chắc chắn đa biến o là một cặp (R, f) , trong đó $R \subseteq \mathbb{R}^m$ là vùng trong đó o được xác định và $f: \mathbb{R}^m \rightarrow \mathbb{R}_0^+$ là hàm mật độ xác suất của o tại mỗi điểm $z \in R$.

Một điểm dữ liệu không chắc chắn đơn biến o là một bộ $(a^{(1)}, \dots, a^{(m)})$. Mỗi thuộc tính $a^{(h)}$ là một cặp $(I^{(h)}, f^{(h)})$, với mỗi $h \in [1 \dots m]$, trong đó $I^{(h)} = [l^{(h)}, u^{(h)}]$ là khoảng xác định của $a^{(h)}$, và $f^{(h)}: \mathbb{R} \rightarrow \mathbb{R}^+$ là hàm mật độ xác suất gán một giá trị xác suất cho mỗi $z \in I^{(h)}$.

Đối với mỗi điểm dữ liệu không chắc chắn đa biến, hàm mật độ xác suất tham gia vào biểu diễn của nó có thể là liên tục hoặc rời rạc. Một hàm mật độ xác suất đa biến m chiều liên tục được xác định trên một vùng $\mathfrak{R} \subseteq \mathbb{R}^m$ là một hàm $f: \mathfrak{R} \rightarrow \mathbb{R}_0^+$ sao cho:

$$\int_{z \in R} f(z) dz = 1 \quad \text{and} \quad \int_{z \in \mathbb{R}^m \setminus R} f(z) dz = 0$$

Một hàm mật độ xác suất đa biến rời rạc m chiều được xác định trên một tập các điểm $S = \{z_1, \dots, z_v\}$ ($z_u \in \mathbb{R}^m$, cho mỗi $u \in [1 \dots v]$) là một hàm $f: \mathbb{R}^m \rightarrow \mathbb{R}_0^+$ sao cho:

$$\sum_{z \in S} f(z) = 1 \quad \text{và} \quad \sum_{z \in \mathbb{R}^m \setminus S} f(z) dz = 0$$

Đối với mô hình đơn biến, một hàm mật độ xác suất liên tục (hoặc rời rạc) đơn biến có thể được xem như một trường hợp đặc biệt của hàm mật độ xác suất đa biến liên tục (hoặc rời rạc), trong đó miền xác định (hoặc tập xác định) là một tập con của miền xác định trong không gian nhiều chiều $\mathbb{R}(i.e., m = 1)$.

2.1.2. Khoảng cách không chắc chắn UK-medoids

Để đo khoảng cách giữa các điểm dữ liệu không chắc chắn, cần thiết phải xây dựng một khái niệm phù hợp về khoảng cách trong điều kiện không chắc chắn gắn liền với thuật toán phân lớp. Khái niệm này được thể hiện thông qua hàm khoảng cách không chắc chắn, cho phép đánh giá mức độ khác biệt giữa các điểm dữ liệu không chính xác.

Nhằm đảm bảo tính tổng quát và độc lập với mô hình biểu diễn sự không chắc chắn, chúng tôi đưa ra các định nghĩa cho hàm khoảng cách không chắc chắn áp dụng cho cả mô hình đơn biến và đa biến.

Khoảng cách không chắc chắn giữa hai điểm dữ liệu o_i và o_j , ký hiệu là $\delta(o_i, o_j)$, được định nghĩa như sau:

$$\delta(o_i, o_j, z) = \int_{x \in R} \Delta(o_i, o_j, z) dz$$

Trong đó $\Delta(o_i, o_j, z)$ là hàm khoảng cách không chắc chắn tại điểm z , được định nghĩa:

$$\Delta(o_i, o_j, z) = \begin{cases} 1 & \text{Nếu } i = j, z = 0 \\ 0 & \text{Nếu } i = j, z \neq 0 \end{cases}$$

Đối với các điểm dữ liệu đa biến không chắc chắn, nếu $o_i = (R_i, f_i)$ và $o_j = (R_j, f_j)$, khoảng cách không chắc chắn $\Delta(o_i, o_j, z)$ được tính như sau:

$$\Delta(o_i, o_j, z) = \int_{x \in R_i} \int_{y \in R_j} I[D(x, y) = z] f_i(x) f_j(y) dx dy$$

Trong đó:

- $D(x, y)$ là khoảng cách Euclide giữa x và y .

- $I[A]$ là hàm đặc trưng, bằng 1 nếu sự kiện A xảy ra, và bằng 0 nếu A không xảy ra.

Tuy nhiên, việc tính tích phân trực tiếp trong biểu thức $\Delta(o_i, o_j, z)$ thường khá phức tạp. Một giải pháp thay thế là xấp xỉ biểu thức này bằng giá trị kỳ vọng của khoảng cách không chắc chắn.

$$\Delta(o_i, o_j) = \int_{x \in R_i} \int_{y \in R_j} D(x, y) f_i(x) f_j(y) dx dy$$

Ví dụ: Xét 3 điểm dữ liệu không chắc chắn có dạng khoảng $[a, b]$:

Điểm	x_1	x_2	x_3
Đoạn $[a, b]$	$[1, 3]$	$[4, 6]$	$[7, 9]$

- Khoảng cách không chắc chắn $\delta(o_i, o_j)$ giữa hai điểm dữ liệu x_i và x_j :

$$\delta(x_i, x_j) = \int_{u \in [a_i, b_i]} \int_{v \in [a_j, b_j]} D(u, v) f_i(u) f_j(v) dv du$$

Trong đó:

+ $D(u, v) = |u - v|$ (vì dữ liệu 1 chiều).

+ $f_i(u), f_j(v)$ là hàm mật độ xác suất của x_i, x_j . Vì $x_i \sim Uniform[a_i, b_i]$, hàm mật độ xác suất là:

$$f_i(u) = \frac{1}{b_i - a_i} \text{ với } u \in [a_i, b_i]$$

- Tính $\delta(x_1, x_2)$:

$$+ x_1 \sim Uniform[1, 3], f_1(u) = \frac{1}{3-1} = \frac{1}{2}.$$

$$+ x_2 \sim Uniform[4, 6], f_2(v) = \frac{1}{6-4} = \frac{1}{2}.$$

$$+ \delta(x_1, x_2) = \int_{u=1}^3 \int_{v=4}^6 |u - v| \frac{1}{2} \frac{1}{2} dv du.$$

+ Vì $u \in [1, 3], v \in [4, 6]$, nên $u < v$, do đó $|u - v| = v - u$.

+ Tích phân:

$$\delta(x_1, x_2) = \int_{u=1}^3 \int_{v=4}^6 (v - u) \frac{1}{4} dv du$$

+ Tính tích phân trong:

$$\begin{aligned}
\int_{v=4}^6 (v-u)dv &= \left[\frac{v^2}{2} - uv \right]_{v=4}^6 \\
&= \left(\frac{6^2}{2} - u \cdot 6 \right) - \left(\frac{4^2}{2} - u \cdot 4 \right) \\
&= (18 - 6u) - (8 - 4u) = 10 - 2u
\end{aligned}$$

+ Tích phân ngoài:

$$\begin{aligned}
\delta(x_1, x_2) &= \int_{u=1}^3 (10 - 2u) \frac{1}{4} du = \frac{1}{4} [10u - u^2]_{u=1}^3 \\
&= \frac{1}{4} ((10 \times 3 - 3^2) - (10 \times 1 - 1^2)) = 3
\end{aligned}$$

Kết quả: $\delta(x_1, x_2) = 3$.

2.1.3. Thuật toán UK-medoids

a) Ý tưởng của thuật toán

Thuật toán UK-medoids (Uncertain K-medoids) là một phiên bản mở rộng của K-medoids để xử lý dữ liệu không chắc chắn. Trong UK-medoids, mỗi điểm dữ liệu được biểu diễn bằng một phân phối xác suất thay vì một giá trị cố định. Khoảng cách giữa các điểm không được tính trực tiếp bằng khoảng cách Euclide, mà được tính dựa trên kỳ vọng của khoảng cách giữa các phân phối xác suất. Bài báo định nghĩa khoảng cách không chắc chắn dựa trên hàm mật độ xác suất PDF của các điểm dữ liệu.

b) Đầu vào và đầu ra

- Đầu vào

+ Tập dữ liệu $D = \{o_1, o_2, \dots, o_n\}$, trong đó mỗi o_i là một điểm dữ liệu không chắc chắn, được biểu diễn bằng phân phối xác suất.

+ Số lượng lớp k .

- Đầu ra: Tập k lớp $\{C_1, C_2, \dots, C_k\}$, trong đó mỗi lớp C_i có một medoid m_i .

c) Tiến trình thực hiện thuật toán

Bước 1: Tính khoảng cách không chắc chắn $\delta(o_i, o_j)$ giữa tất cả các cặp điểm dữ liệu $o_i, o_j \in D$.

Bước 2: Khởi tạo medoids bằng cách chọn ngẫu nhiên k điểm dữ liệu từ D để tạo tập hợp medoids ban đầu $S = \{m_1, m_2, \dots, m_k\}$.

Bước 3: Lặp lại cho đến khi medoids không thay đổi:

- Lưu tập medoids hiện tại vào $S' \leftarrow S$.
- Đặt $S' \leftarrow \emptyset$ (tập medoids mới sẽ được xây dựng lại).
- Khởi tạo tập hợp các lớp $C = \{C_1, C_2, \dots, C_k\} \leftarrow \{\emptyset, \emptyset, \emptyset, \dots, \emptyset\}$.
- Gán điểm dữ liệu vào lớp: Với mỗi điểm dữ liệu $o \in D$:
- + Tìm medoid $m_j \in S'$ sao cho khoảng cách $\delta(o, m_j)$ là nhỏ nhất:

$$m_j = \operatorname{argmin}_{d \in S'} \delta(o, d)$$

- + Gán o vào lớp: $C_j: C_j \leftarrow C_j \cup \{o\}$.
- Cập nhật medoids: Với mỗi lớp $C_i \in C$:
- + Tìm điểm dữ liệu $m \in C_i$ sao cho tổng khoảng cách từ m đến tất cả các điểm dữ liệu khác trong C_i là nhỏ nhất: $m = \operatorname{argmin}_{o \in C_i} \sum_{o' \in C_i} \delta(o, o')$
- + Thêm m vào tập medoids mới: $S \leftarrow S \cup \{m\}$.
- Lặp lại cho đến khi $S = S'$ (tập medoids không thay đổi).

Bước 4: Trả về tập các lớp C .

d) Độ phức tạp của thuật toán:

- Thời gian:
 - + Tính khoảng cách ban đầu: $O(n^2T)$, trong đó T là thời gian để tính một tích phân $\delta(o_i, o_j)$.
 - + Khởi tạo medoids: $O(k)$.
 - + Gán điểm dữ liệu vào lớp: $O(nkT)$ cho mỗi lần lặp.
 - + Cập nhật medoids: $O(n^2kT)$ (vì cần tính tổng khoảng cách cho mỗi điểm dữ liệu trong lớp).
 - + Tổng độ phức tạp: $O(n^2T + I \cdot n^2kT)$, trong đó I là số lần lặp.
- Không gian: $O(n^2)$ lưu trữ ma trận khoảng cách và $O(n)$ lưu trữ các lớp.

2.2. Mô hình phân lớp UK-means

2.2.1. Bài toán phân lớp không chắc chắn UK-means

Gọi S là tập hợp gồm các vector v -chiều x_i , với $i=1$ đến n , biểu diễn cho các giá trị thuộc tính của tất cả các bản ghi trong bài toán phân cụm. Mỗi bản ghi o_i được gán với một hàm mật độ xác suất (PDF) $f_i(x)$, biểu diễn phân phối xác suất của các giá trị thuộc tính của o_i tại thời điểm t . Bài toán phân lớp đặt mục tiêu tìm một tập hợp các cụm $C = \{C_1, C_2, \dots, C_k\}$, trong đó mỗi cụm C_j có một giá

trị trung bình đại diện, được xác định dựa trên mức độ tương đồng giữa các điểm dữ liệu. Mặc dù các thuật toán phân lớp khác nhau có thể sử dụng các hàm mục tiêu khác nhau, mục tiêu chung vẫn là tối thiểu hóa khoảng cách giữa các điểm dữ liệu trong cùng một cụm và tối đa hóa khoảng cách giữa các điểm dữ liệu thuộc các cụm khác nhau. Việc tối thiểu hóa khoảng cách nội cụm cũng có thể được hiểu là giảm khoảng cách giữa mỗi vector dữ liệu x_i và trung bình cụm C_j của cụm C_j mà x_i được gán vào. Để xem xét sự không chính xác của dữ liệu trong quá trình phân cụm, chúng tôi đề xuất một thuật toán phân cụm với mục tiêu giảm thiểu tổng bình phương sai số kỳ vọng $E(SSE)$. Lưu ý rằng một điểm dữ liệu x_i được xác định bởi một vùng không chính xác với một hàm PDF không chính xác $f(x_i)$.

Với một tập các cụm C_j , tổng bình phương sai số kỳ vọng được tính như sau:

$$E \left(\sum_{j=1}^k \sum_{i \in C_j} \|c_j - x_i\|^2 \right) = \sum_{j=1}^k \sum_{i \in C_j} \int \|c_j - x_i\|^2 f(x_i) dx_i$$

Trong đó, $\| \cdot \|$ là một độ đo khoảng cách giữa một điểm dữ liệu x_i và trung bình cụm C_j . Trung bình cụm được cho bởi:

$$c_j = E \left(\frac{1}{|C_j|} \sum_{i \in C_j} x_i \right) = \frac{1}{|C_j|} \sum_{i \in C_j} \int x_i f(x_i) dx_i$$

2.2.2. Thuật toán UK-means

a) Ý tưởng của thuật toán

Thuật toán UK-Means (Uncertain K-Means) là một sự mở rộng của K-Means, cho phép xử lý trực tiếp dữ liệu không chắc chắn bằng cách thay thế khoảng cách Euclid thông thường bằng khoảng cách kỳ vọng (Expected Distance) giữa một điểm dữ liệu không chắc chắn và tâm lớp. Cách tiếp cận này giúp cải thiện độ chính xác trong việc phân lớp, đặc biệt khi dữ liệu có nhiễu hoặc độ tin cậy thấp.

b) Đầu vào và đầu ra

- Đầu vào

+ Tập dữ liệu $D = \{o_1, o_2, \dots, o_n\}$, trong đó mỗi o_i là một điểm dữ liệu không chắc chắn, được biểu diễn bằng phân phối xác suất.

+ Số lượng lớp k .

- Đầu ra: Tập k lớp $\{C_1, C_2, \dots, C_k\}$.

c) Tiến trình thực hiện thuật toán UK-means

Bước 1: Chọn k điểm làm tâm lớp ban đầu.

Bước 2: Lặp lại cho đến khi hội tụ:

- Gán lớp: Gán mỗi điểm dữ liệu không chắc chắn cho lớp có khoảng cách kỳ vọng đến tâm lớp nhỏ nhất. Với mỗi điểm dữ liệu x_i ($i = 1$ đến n),

+ Tính khoảng cách kỳ vọng $E(||c_j - x_j||)$ đến tất cả các trung bình lớp c_j .

+ Gán x_i vào lớp C_j có khoảng cách kỳ vọng nhỏ nhất.

- Cập nhật trung bình lớp: Với mỗi lớp C_j ($j = 1$ đến K), tính lại trung bình lớp bằng cách lấy giá trị trung bình của tất cả các điểm dữ liệu (sử dụng lại giá trị trung bình của phân phối) thuộc lớp C_j với $\mu_j = \frac{1}{|C_j|} \times \sum_{x \in C_j} x$.

Bước 3: Điều kiện dừng

- Lặp lại Bước 2 cho đến khi đạt điều kiện hội tụ.

- Điều kiện hội tụ có thể được xác định theo một trong các tiêu chí sau:

+ Không còn sự thay đổi trong việc gán lớp cho các điểm dữ liệu.

+ Sự thay đổi vị trí các tâm cụm (trung bình cụm) nhỏ hơn một ngưỡng xác định trước.

+ Đã thực hiện số vòng lặp tối đa cho phép.

Bước 4: Trả về tập các lớp $C = \{C_1, C_2, \dots, C_k\}$ và các trung bình lớp tương ứng $c_1, c_2, \dots, c_k$.

CHƯƠNG 3: CÀI ĐẶT THỰC NGHIỆM

3.1. Cài đặt thuật toán UK-medoids

3.1.1. Bộ dữ liệu

a) Nguồn dữ liệu:

Dữ liệu được lấy từ kho lưu trữ UCI Machine Learning Repository, thông qua gói `ucimlrepo` trong Python [6]. Cụ thể, chúng tôi sử dụng 2 tập dữ liệu phổ biến:

- **Iris**: Tập dữ liệu về hoa Iris [9]
- **Wine**: Tập dữ liệu về rượu vang [10]

Các tập dữ liệu này được tải trực tiếp bằng hàm `fetch_ucirepo` từ gói `ucimlrepo`, không cần tải file CSV từ web hay đọc từ nguồn cục bộ.

b) Đọc dữ liệu:

- Sử dụng gói `ucimlrepo` để tải các tập dữ liệu Iris (ID=53), Wine (ID=109).
- Các tập dữ liệu được lưu vào biến `iris` và `wine`.

c) Tiền xử lý dữ liệu:

1. Lọc và chuẩn hóa nhãn:

- Với tập **Iris**, nhãn gốc là chuỗi (ví dụ: "Iris-setosa", "cp"), nên cần ánh xạ thành số nguyên:
 - Iris: {'Iris-setosa': 0, 'Iris-versicolor': 1, 'Iris-virginica': 2}.
- Với tập **Wine**, nhãn đã là số nguyên, nhưng code kiểm tra để đảm bảo tính hợp lệ.

2. Chuẩn hóa đặc trưng:

Sử dụng `StandardScaler` từ gói `sklearn.preprocessing` [7] để chuẩn hóa dữ liệu, đảm bảo các thuộc tính có trung bình bằng 0 và độ lệch chuẩn bằng 1. Điều này giúp giảm ảnh hưởng của các thang đo khác nhau giữa các thuộc tính.

3. Chuyển đổi thành dữ liệu không chắc chắn:

- Mỗi mẫu dữ liệu ban đầu là một vector giá trị cố định (deterministic). Để tạo dữ liệu không chắc chắn (uncertain data), code sử dụng các phân phối xác suất (PDF) để mô phỏng tính không chắc chắn:
 - **UniformMV** (phân phối đều đa biến): Tạo đoạn $[a, b][a, b][a, b]$ cho mỗi thuộc tính, với độ dài đoạn được xác định dựa trên khoảng giá trị của thuộc tính và một tỷ lệ (percentage) cụ thể cho từng tập dữ liệu.

- **Normal** (phân phối chuẩn đa biến): Tạo phân phối chuẩn với trung bình là giá trị gốc và phương sai được tính dựa trên khoảng giá trị của thuộc tính.
- **Binomial** (phân phối đều đơn biến): Tạo đoạn $[a, b][a, b][a, b]$ cho mỗi thuộc tính, tương tự như UniformMV nhưng áp dụng riêng lẻ cho từng thuộc tính.
- Tỷ lệ mở rộng đoạn (percentage) được điều chỉnh riêng cho từng tập dữ liệu và từng loại phân phối

Sau khi tiền xử lý, các tập dữ liệu có đặc điểm như sau:

Tập dữ liệu	Số mẫu	Số thuộc tính	Số lớp	Mô tả nhãn/lớp
Iris	150	4	3	0 (setosa), 1 (versicolor), 2 (virginica)
Wine	178	13	3	1, 2, 3 (ba loại rượu vang)

Bảng 3.1. Mô tả dữ liệu thu được sau khi tiền xử lý

- **Dạng dữ liệu:** Sau khi chuyển đổi, mỗi thuộc tính của mỗi mẫu không phải là một giá trị cố định, mà được biểu diễn bằng một phân phối xác suất (UniformMV, Normal, hoặc Binomial) [8], tùy thuộc vào tham số pdf_name được chọn trong quá trình thử nghiệm.
- **Nhãn/lớp:** Mặc dù các tập dữ liệu có nhãn, thuật toán UK-medoids là một thuật toán **không giám sát** (unsupervised), nên nhãn chỉ được sử dụng để đánh giá kết quả phân lớp, không tham gia vào quá trình xây dựng mô hình.

3.1.2. Xây dựng mô hình

1. Import các thư viện hỗ trợ

```
import time
import warnings
from ucimlrepo import fetch_ucirepo
import numpy as np
from sklearn.preprocessing import StandardScaler
from scipy.integrate import quad, dblquad
from scipy.stats import multivariate_normal
from collections import defaultdict
import traceback
```

```
from pyclustering.cluster.kmedoids import kmedoids
from pyclustering.utils import calculate_distance_matrix
import matplotlib.pyplot as plt
```

2. Cấu hình ban đầu: Bỏ qua các cảnh báo và cố định seed ngẫu nhiên để đảm bảo tính tái lập

```
warnings.filterwarnings("ignore", category=RuntimeWarning)
warnings.filterwarnings("ignore", category=DeprecationWarning)
warnings.filterwarnings("ignore", category=UserWarning)

# Set random seed for reproducibility
np.random.seed(42)
```

3. Tải và tiền xử lý dữ liệu

```
# --- Data Loading and Preprocessing ---
iris = fetch_ucirepo(id=53)
wine = fetch_ucirepo(id=109)

iris_label_map = {'Iris-setosa': 0, 'Iris-versicolor': 1, 'Iris-
virginica': 2}
iris_targets = np.array([iris_label_map[label] for label in
iris.data.targets.values.ravel()])

wine_targets = wine.data.targets.values.ravel().astype(int)

# Lưu dữ liệu gốc (chưa chuẩn hóa) để sử dụng cho trực quan hóa
iris_data_original = iris.data.features.values # Đã sửa lỗi cú pháp
wine_data_original = wine.data.features.values

# Chuẩn hóa dữ liệu để sử dụng trong thuật toán phân cụm
scaler = StandardScaler()
datasets = {
    "Iris": scaler.fit_transform(iris_data_original),
    "Wine": scaler.fit_transform(wine_data_original),
}

# Lưu dữ liệu gốc để trực quan hóa
original_data = {
    "Iris": iris_data_original,
    "Wine": wine_data_original,
```

```

}

true_labels = {
    "Iris": iris_targets,
    "Wine": wine_targets,
}

k_values = {name: len(np.unique(targets)) for name, targets in
true_labels.items()}
# Tên các đặc trưng của Iris
feature_names = {
    "Iris": ["Sepal Length", "Sepal Width", "Petal Length", "Petal
Width"],
    "Wine": [f"Feature {i+1}" for i in range(datasets["Wine"].shape[1])]
}

```

4. Định nghĩa các lớp cốt lõi

```

# --- Core Classes (Entry, Dataset, Classification) ---
class Entry:
    def __init__(self, id_, class_label_id, values, u_values,
original_values=None):
        self.id_ = id_
        self.class_label_id = int(class_label_id)
        self.values = np.array(values) # Giá trị đã chuẩn hóa
        self.u_values = u_values
        self.original_values = np.array(original_values) if
original_values is not None else np.array(values) # Giá trị gốc
        def get_id(self): return self.id_
        def get_values(self): return self.values
        def get_original_values(self): return self.original_values
        def set_pdf(self, pdf, index):
            if not isinstance(self.u_values, list): self.u_values = [None] *
len(self.values)
            if index < len(self.u_values): self.u_values[index] = pdf
            else: raise IndexError(f"Index {index} out of bounds")
        def get_random_sample(self):
            if self.u_values is None: return self.values
            if isinstance(self.u_values, list):
                sample = [pdf.get_random_sample() if pdf is not None else
self.values[idx] for idx, pdf in enumerate(self.u_values)]
                return np.array(sample)
            else: return np.array(self.u_values.get_random_sample())

```

```

class Dataset:
    def __init__(self, entries): self.entries = entries
    def get_size(self): return len(self.entries)
    def get_entry(self, i): return self.entries[i]

class Classification:
    def __init__(self, objects): self.labels = [int(obj.class_label_id)
for obj in objects]
    def get_label(self, i): return self.labels[i]

```

5. Xây dựng các lớp hàm mật độ xác suất

```

#--- PDF Classes (MultivariateUniform, MultivariateGaussian, Binomial) ---
-
class MultivariateUniform:
    def __init__(self, min_values, max_values, n_samples=None):
        self.min_values = np.array(min_values)
        self.max_values = np.array(max_values)
        self.dim = len(min_values)
        self.volume = np.prod([(max_val - min_val) if (max_val > min_val)
else 1e-9 for min_val, max_val in zip(self.min_values, self.max_values)])
        self.pdf_value = 1.0 / self.volume if self.volume > 1e-12 else 0
    def pdf(self, x):
        x = np.array(x)
        if x.shape != self.min_values.shape: return 0.0
        if self.volume <= 1e-12: return 0.0
        return self.pdf_value if np.all((x >= self.min_values) & (x <=
self.max_values)) else 0.0
    def get_random_sample(self): return
np.random.uniform(self.min_values, self.max_values, size=self.dim)

class MultivariateGaussian:
    def __init__(self, min_values, max_values, n_samples=None, rho=0.05):
        self.min_values = np.array(min_values)
        self.max_values = np.array(max_values)
        self.rho = max(1e-6, min(1.0, rho))
        self.mu = (self.min_values + self.max_values) / 2.0
        self.dim = len(min_values)
        variances = [(max_val - min_val)**2 / 12.0 if max_val > min_val
else 1e-6 for min_val, max_val in zip(self.min_values, self.max_values)]
        self.sigma = np.diag(variances) * self.rho

```



```

        self.sigma += np.eye(self.dim) * 1e-9
    try:
        np.linalg.cholesky(self.sigma)
        self._scipy_pdf_func = multivariate_normal(mean=self.mu,
cov=self.sigma, allow_singular=False)
    except np.linalg.LinAlgError:
        diag_sigma = np.maximum(np.diag(self.sigma), 1e-6)
        self.sigma = np.diag(diag_sigma)
    try:
        np.linalg.cholesky(self.sigma)
        self._scipy_pdf_func = multivariate_normal(mean=self.mu,
cov=self.sigma, allow_singular=False)
    except np.linalg.LinAlgError:
        self.sigma = np.eye(self.dim) * 0.1
        self._scipy_pdf_func = multivariate_normal(mean=self.mu,
cov=self.sigma, allow_singular=False)

    def pdf(self, x):
        x = np.array(x)
        if x.shape != self.mu.shape: return 0.0
        try: return self._scipy_pdf_func.pdf(x)
        except: return 0.0

    def get_random_sample(self):
        try: return np.random.multivariate_normal(self.mu, self.sigma)
        except np.linalg.LinAlgError: return self.mu

class Binomial:
    def __init__(self, lower_bound, upper_bound, value=None,
n_samples=None):
        self.lower_bound = lower_bound
        self.upper_bound = upper_bound
        if self.upper_bound <= self.lower_bound: self.upper_bound =
self.lower_bound + 1e-9
        self.range = self.upper_bound - self.lower_bound
        self.pdf_value = 1.0 / self.range if self.range > 1e-12 else 0.0
    def pdf(self, x):
        if self.range <= 1e-12: return 0.0
        return self.pdf_value if self.lower_bound <= x <=
self.upper_bound else 0.0
    def get_random_sample(self): return
np.random.uniform(self.lower_bound, self.upper_bound)

```

6. Tính khoảng cách không chắc chắn

```
# --- Analytical Distance Calculation ---
class AnalyticalDistanceCalculator:
    def __init__(self, dataset):
        self.dataset = dataset
        self.dim = len(dataset.get_entry(0).get_values()) if
dataset.get_size() > 0 else 0
        self._mc_fallback_count = 0
        self._total_distance_calcs = 0
        self._int_opts = {'epsabs':1.49e-06, 'epsrel':1.49e-06}
        self._gaussian_bound_stddevs = 6
    def compute_distance_matrix(self, dpm):
        n = self.dataset.get_size()
        if n == 0: return np.array([])
        distance_matrix = np.zeros((n, n))
        start_time = time.time()
        self._mc_fallback_count = 0; self._total_distance_calcs = 0
        for i in range(n):
            e1 = self.dataset.get_entry(i)
            row_distances = []
            for j in range(i + 1):
                if i == j: distance = 0.0
                else:
                    e2 = self.dataset.get_entry(j)
                    try: distance = self._compute_pairwise_distance(e1,
e2)
                    except Exception as e:
                        distance = max(1e-6,
np.linalg.norm(e1.get_values() - e2.get_values()))
                    if not np.isfinite(distance) or distance < 0:
                        distance = max(1e-6, np.linalg.norm(e1.get_values()
- e2.get_values()))
                    distance_matrix[i, j] = distance; distance_matrix[j, i] =
distance
                    row_distances.append(distance)
            dpm.write_data_row(row_distances[:i+1])
            if (i + 1) % (n // 10 if n > 50 else 50) == 0:
                elapsed = time.time() - start_time
                rate = (i + 1) / elapsed if elapsed > 0 else 0

        if self._total_distance_calcs > 0:
```

```

        mc_percentage = (self._mc_fallback_count /
self._total_distance_calcs) * 100
        return distance_matrix

    def _compute_pairwise_distance(self, e1, e2):
        self._total_distance_calcs += 1
        u1 = e1.u_values; u2 = e2.u_values; expected_sq_dist = -1.0
        if u1 is not None and u2 is not None:
            if isinstance(u1, MultivariateUniform) and isinstance(u2,
MultivariateUniform): expected_sq_dist =
self._uniform_uniform_sq_distance(u1, u2)
            elif isinstance(u1, MultivariateGaussian) and isinstance(u2,
MultivariateGaussian): expected_sq_dist =
self._gaussian_gaussian_sq_distance(u1, u2)
            elif isinstance(u1, list) and isinstance(u2, list):
expected_sq_dist = self._univariate_list_sq_distance(u1, u2)
            else: expected_sq_dist =
self._numerical_integration_sq_distance(e1, e2)
            elif u1 is not None and u2 is None: expected_sq_dist =
self._pdf_point_sq_distance(u1, e2.get_values())
            elif u1 is None and u2 is not None: expected_sq_dist =
self._pdf_point_sq_distance(u2, e1.get_values())
            else: expected_sq_dist = np.sum((e1.get_values() -
e2.get_values())**2)
            if expected_sq_dist < -1e-9: raise ValueError(f"Sq dist calc
failed pair ({e1.id_}, {e2.id_})")
            return np.sqrt(max(0, expected_sq_dist))

    def _uniform_uniform_sq_distance(self, u1: MultivariateUniform, u2:
MultivariateUniform):
        if u1.dim != u2.dim: raise ValueError("Dim mismatch")
        ex1=(u1.min_values+u1.max_values)/2.0;
ex2=(u2.min_values+u2.max_values)/2.0
        r1=u1.max_values-u1.min_values; r2=u2.max_values-u2.min_values
        v1=np.where(r1>1e-12, (r1**2)/12.0,0); v2=np.where(r2>1e-
12, (r2**2)/12.0,0)
        return np.sum((ex1-ex2)**2+v1+v2)

    def _gaussian_gaussian_sq_distance(self, u1: MultivariateGaussian,
u2: MultivariateGaussian):
        if u1.dim != u2.dim: raise ValueError("Dim mismatch")

```

```

        return np.sum((u1.mu-
u2.mu)**2)+np.trace(u1.sigma)+np.trace(u2.sigma)

    def _univariate_list_sq_distance(self, u1_list, u2_list):
        if len(u1_list) != len(u2_list): raise ValueError("Dim mismatch")
        total_sq_dist = 0
        for d in range(len(u1_list)):
            p1=u1_list[d]; p2=u2_list[d]
            if isinstance(p1, Binomial) and isinstance(p2, Binomial):
                a1,b1=p1.lower_bound,p1.upper_bound;
a2,b2=p2.lower_bound,p2.upper_bound
                r1=b1-a1; r2=b2-a2; ex1=(a1+b1)/2.0; ex2=(a2+b2)/2.0
                v1=(r1**2)/12.0 if r1>1e-12 else 0; v2=(r2**2)/12.0 if
r2>1e-12 else 0
                total_sq_dist += (ex1-ex2)**2+v1+v2
            else:
                integrand = lambda x, y: (x-y)**2 * p1.pdf(x) *
p2.pdf(y)
                b1=self._get_1d_bounds(p1); b2=self._get_1d_bounds(p2)
                try: val, err = dblquad(integrand, b1[0],b1[1], lambda
y: b2[0], lambda y: b2[1], **self._int_opts); total_sq_dist += val
            except:
                s1=[p1.get_random_sample() for _ in range(1000)];
s2=[p2.get_random_sample() for _ in range(1000)]
                total_sq_dist += np.mean([(x-y)**2 for x,y in
zip(s1,s2)])
        return total_sq_dist

    def _pdf_point_sq_distance(self, u_pdf, point_y):
        point_y = np.array(point_y)
        if isinstance(u_pdf, MultivariateGaussian): return
np.sum((u_pdf.mu-point_y)**2)+np.trace(u_pdf.sigma)
        elif isinstance(u_pdf, MultivariateUniform):
            ex=(u_pdf.min_values+u_pdf.max_values)/2.0;
r=u_pdf.max_values-u_pdf.min_values
            vx=np.where(r>1e-12, (r**2)/12.0,0); return np.sum((ex-
point_y)**2+vx)
        elif isinstance(u_pdf, list):
            total_sq_dist = 0
            for d, p in enumerate(u_pdf):
                yd=point_y[d]
                if isinstance(p, Binomial):

```

```

        a,b=p.lower_bound,p.upper_bound; r=b-a
        ex=(a+b)/2.0; vx=(r**2)/12.0 if r>1e-12 else 0
        total_sq_dist += (ex-yd)**2+vx
    else:
        integrand = lambda x: (x-yd)**2 * p.pdf(x);
bnds=self._get_1d_bounds(p)
        try: val, err = quad(integrand, bnds[0], bnds[1],
**self._int_opts); total_sq_dist += val
        except:
            s=[p.get_random_sample() for _ in range(1000)];
total_sq_dist += np.mean([(x-yd)**2 for x in s])
        return total_sq_dist
    else:
        self._mc_fallback_count += 1; pdf_entry =
Entry(None,None,None,u_pdf); crisp_entry = Entry(None,None,point_y,None)
        return self._monte_carlo_sq_distance(pdf_entry, crisp_entry)

def _numerical_integration_sq_distance(self, e1, e2):
    self._mc_fallback_count += 1
    return self._monte_carlo_sq_distance(e1, e2)

def _monte_carlo_sq_distance(self, e1, e2, n_samples=10000):
    s1 = np.array([e1.get_random_sample() for _ in range(n_samples)])
    s2 = np.tile(e2.get_values(), (n_samples,1)) if e2.u_values is
None else np.array([e2.get_random_sample() for _ in range(n_samples)])
    return np.mean(np.sum((s1-s2)**2, axis=1))

def _get_1d_bounds(self, pdf):
    dr=self._gaussian_bound_stddevs
    if isinstance(pdf, Binomial): m=(pdf.upper_bound-
pdf.lower_bound)*0.01; return [pdf.lower_bound-m, pdf.upper_bound+m]
    elif isinstance(pdf, MultivariateGaussian) and pdf.dim==1:
sd=np.sqrt(max(1e-12, pdf.sigma[0,0])); return [pdf.mu[0]-dr*sd,
pdf.mu[0]+dr*sd]
    else: return [-dr, dr]

def _get_nd_bounds(self, pdf):
    dr=self._gaussian_bound_stddevs
    if isinstance(pdf, MultivariateUniform):
        return list(zip(pdf.min_values, pdf.max_values))
    elif isinstance(pdf, MultivariateGaussian):

```

```

        sd=np.sqrt(np.maximum(1e-12, np.diag(pdf.sigma))); return
list(zip(pdf.mu-dr*sd, pdf.mu+dr*sd))
        else: raise TypeError(f"Cannot determine N-D bounds for
{type(pdf)}")

```

7. Tải dữ liệu với TUNED Uncertainty Generation

```

# --- Data Loading with TUNED Uncertainty Generation ---
class DataLoader:
    def __init__(self, data, labels):
        self.n_samples = 100; self.binomial_samples = 100
        self.data = data; self.original_labels = labels
        self.mapped_labels = self._map_labels(labels)
        self.d = None; self.classification = None; self.min_max = None
        self.data_attributes = data.shape[1]; self.load_data()

    def _map_labels(self, labels):
        unique_labels = sorted(list(set(labels))); label_map = {l: i for
i, l in enumerate(unique_labels)}
        return np.array([label_map[l] for l in labels])

    def load_data(self):
        objs = [Entry(str(i), self.mapped_labels[i], self.data[i], None)
for i in range(len(self.data))]
        self.d = Dataset(objs); self.classification =
Classification(objs)

    def generate_uncertainty(self, pdf_name, bound_type="random",
dataset_name="Unknown"):
        if bound_type != "random": raise NotImplementedError
        #print(f"Generating '{pdf_name}' uncertainty for {dataset_name}
(Tuned)...")
        self.compute_attributes_range()

        # --- AGGRESSIVE TUNING PARAMETERS (Trial 2 Strategy) ---
        perc_boost_factor = 1.0
        rho_val = 0.25

        # High base percentages aimed at targets
        percentage_uniformMV_map = {
            'Iris': 0.40,
            'Wine': 0.03,
        }
        percentage_binomial_map = {

```

```

        'Iris': 0.30,
        'Wine': 0.15,
    }
    # Adjust Normal percentages specifically
    percentage_normal_map = {
        'Iris': 0.15,
        'Wine': 0.26,
    }
    rho_map = {'Iris': rho_val, 'Wine': rho_val, 'Glass': rho_val,
'Ecoli': rho_val}
    # --- END TUNING PARAMETERS ---

    pdf_name_lower = pdf_name.lower()
    percentage = 0.0
    if pdf_name_lower == "uniformmv": percentage =
percentage_uniformMV_map.get(dataset_name, 0.15) # Default base
    elif pdf_name_lower == "binomial": percentage =
percentage_binomial_map.get(dataset_name, 0.15) # Default base
    elif pdf_name_lower == "normal": percentage =
percentage_normal_map.get(dataset_name, 0.15) # Default base
    else: raise ValueError(f"Unknown PDF: {pdf_name}")
    rho = rho_map.get(dataset_name, rho_val)

    missing_ranges = 0
    for i in range(self.d.get_size()):
        entry = self.d.get_entry(i); class_id = entry.class_label_id;
orig_vals = entry.get_values()
        class_ranges = self.min_max.get(class_id)
        if class_ranges is None: # Fallback if class range missing
            missing_ranges += 1; class_ranges = [[v-0.1, v+0.1] for v
in orig_vals]
        if len(class_ranges) != self.data_attributes: class_ranges =
[[v-0.1, v+0.1] for v in orig_vals] # Ensure correct dim

        min_b, max_b = [], []
        for j in range(self.data_attributes):
            min_v, max_v = class_ranges[j] if
isinstance(class_ranges[j], (list, tuple)) and len(class_ranges[j])==2 else
(orig_vals[j], orig_vals[j])
            r = max_v - min_v; offset = max(0, r*percentage) + (0.01
if r<=1e-6 else 0)
            c = orig_vals[j]; lb = c-offset; ub = c+offset

```

```

        if ub <= lb: ub = lb + 1e-6
        min_b.append(lb); max_b.append(ub)

        if pdf_name_lower == "normal": entry.u_values =
MultivariateGaussian(min_b, max_b, self.n_samples, rho)
        elif pdf_name_lower == "uniformmv": entry.u_values =
MultivariateUniform(min_b, max_b, self.n_samples)
        elif pdf_name_lower == "binomial": entry.u_values =
[Binomial(lb,ub,orig_vals[j],self.binomial_samples) for j,(lb,ub) in
enumerate(zip(min_b,max_b))]

    def compute_attributes_range(self):
        tmp = defaultdict(lambda: [[float('inf')]*self.data_attributes,
[float('-inf')]*self.data_attributes])
        for entry in self.d.entries:
            cid=entry.class_label_id; vals=entry.get_values();
mins,maxs=tmp[cid]
            for j,v in enumerate(vals): mins[j]=min(mins[j],v);
maxs[j]=max(maxs[j],v)
            self.min_max={cid: [[min(mn[j],mx[j]),max(mn[j],mx[j])]] for j in
range(self.data_attributes)] for cid,(mn,mx) in tmp.items()}

    def get_dataset(self): return self.d
    def get_classification(self): return self.classification

```

8. Xử lý tệp ma trận khoảng cách

```

# --- Distance Matrix File Handling ---
class DistanceMatrixFile:
    def __init__(self, fp): self.fp=fp; self.f=None
    def write_data_row(self, dr):
        if self.f is None: self.f=open(self.fp,"w")
        self.f.write(" ".join(map(str, dr))+ "\n")
    def close(self):
        if self.f: self.f.close(); self.f=None
    def load_matrix(self, n):
        # print(f>Loading distance matrix from {self.fp}...")
        dm = np.zeros((n,n))
        try:
            with open(self.fp,"r") as f:
                for i in range(n):
                    ln=f.readline().strip();
                    if not ln: raise ValueError(f"Empty line row {i+1}")

```



```

        rd=list(map(float,ln.split()))
        if len(rd)!=i+1: raise ValueError(f"Row {i+1} len
{len(rd)}!=expected {i+1}")
        dm[i,:i+1]=rd
        li=np.tril_indices(n,k=-1); dm[li[1],li[0]]=dm[li]
        np.fill_diagonal(dm,0)
        # print("Matrix loaded.")
        return dm
    except FileNotFoundError: print(f"Error: File {self.fp} not
found."); raise
    except Exception as e: print(f"Error loading matrix: {e}"); raise

```

9. Thuật toán UK-medoids

```

--- UK-Medoids Algorithm ---
class UKMedoids:
    def __init__(self, dataset):
        self.d = dataset; self.dist = None; self.n = dataset.get_size()
    def compute_multivariate_distances(self, dpm):
        calc = AnalyticalDistanceCalculator(self.d); self.dist =
calc.compute_distance_matrix(dpm)
    def load_distances(self, fp):
        dpm = DistanceMatrixFile(fp); self.dist = dpm.load_matrix(self.n)
    def _init_random(self, k): return np.random.choice(self.n, k,
replace=False)
    def _init_ff(self, k): # Farthest First
        if k==0: return np.array([],dtype=int)
        if self.dist is None: raise ValueError("Distances needed")
        meds=np.zeros(k,dtype=int); meds[0]=np.random.randint(0,self.n)
        if k==1: return meds
        sel=np.zeros(self.n,dtype=bool); sel[meds[0]]=True
        for i in range(1,k):
            c_idx=meds[:i]; cand_idx=np.where(~sel)[0]
            if not cand_idx.size: return meds[:i]
            sums=np.sum(self.dist[cand_idx][:,c_idx],axis=1)
            nxt=cand_idx[np.argmax(sums)]; meds[i]=nxt; sel[nxt]=True
        return meds
    def k_medoids(self, k, max_iter=100, init_method='farthest_first'):
        if self.dist is None: raise ValueError("Distances not loaded")
        if k>self.n: raise ValueError(f"k={k}>n={self.n}")
        if k==0: return np.array([]), np.array([])

```

```

        if init_method=='random': c_meds=self._init_random(k)
        elif init_method=='farthest_first': c_meds=self._init_ff(k)
        else: raise ValueError(f"Unknown init: {init_method}")

        lbls=-np.ones(self.n,dtype=int); conv=False;
best_cost=float('inf')
        best_meds=np.copy(c_meds); best_lbls=np.copy(lbls)

        for it in range(max_iter):
            d_meds=self.dist[:,c_meds]; n_lbls=np.argmin(d_meds,axis=1)
            c_cost=np.sum(np.min(d_meds,axis=1))
            cost_imp=c_cost<best_cost-1e-9;
assign_stable=np.array_equal(n_lbls,lbls)

            if not cost_imp and assign_stable and it>0: conv=True; break
            if cost_imp: best_cost=c_cost; best_meds=np.copy(c_meds);
best_lbls=np.copy(n_lbls)
            lbls=n_lbls

            n_meds=np.copy(c_meds); changed=False
            for ci in range(k):
                pts_idx=np.where(lbls==ci)[0]
                if not pts_idx.size: continue
                if len(pts_idx)==1: p_med=pts_idx[0]
                else:
p_med=pts_idx[np.argmin(np.sum(self.dist[np.ix_(pts_idx,pts_idx)],axis=1)
)]

                if p_med!=n_meds[ci]: n_meds[ci]=p_med; changed=True

            if not changed and it>0:
                conv=True
                f_dists=self.dist[:,c_meds];
f_lbls=np.argmin(f_dists,axis=1); f_cost=np.sum(np.min(f_dists,axis=1))
                if f_cost<best_cost: best_cost=f_cost;
best_meds=np.copy(c_meds); best_lbls=f_lbls
                break
            c_meds=n_meds
            # if not conv: print(f"KMedoids max_iter {max_iter} reached.")
            return best_lbls, best_meds

    def execute(self, fp, k, init_method='farthest_first'):

```

```

        self.load_distances(fp); labels, meds = self.k_medoids(k,
init_method=init_method)
        return labels

```

10. Đánh giá kết quả

```

# --- Evaluation Metric ---
class ClusteringExternalEvaluation:
    def __init__(self, truelbls, predlbls):
        self.t=np.array(truelbls); self.p=np.array(predlbls);
self.n=len(self.t)
        self.tp,self.fp,self.fn,self.tn=(0,0,0,0) if self.n==0 or
len(self.p)!=self.n else self._pairs()
    def _pairs(self):
        tp=fp=fn=tn=0
        for i in range(self.n):
            for j in range(i+1,self.n):
                st=self.t[i]==self.t[j]; sp=self.p[i]==self.p[j]
                if st and sp: tp+=1
                elif not st and sp: fp+=1
                elif st and not sp: fn+=1
                else: tn+=1
        return tp,fp,fn,tn
    def precision(self): d=self.tp+self.fp; return self.tp/d if d>0 else
0
    def recall(self): d=self.tp+self.fn; return self.tp/d if d>0 else 0
    def f_measure(self): p=self.precision(); r=self.recall(); d=p+r;
return 2*(p*r)/d if d>0 else 0.0

```

11. Trực quan hóa dữ liệu

```

# --- Visualization Function ---
def visualize_uncertainty(dataset_name, pdf_name, data_loader):
    """
    Trực quan hóa dữ liệu chỉ với các điểm trung tâm (center points) cho
    từng cặp đặc trưng,
    sử dụng giá trị gốc (chưa chuẩn hóa).

    Parameters:
    - dataset_name: Tên tập dữ liệu (Iris, Wine)
    - pdf_name: Loại PDF (UniformMV, Normal, Binomial)
    - data_loader: Đối tượng DataLoader chứa dữ liệu
    """

```

```

dataset = data_loader.get_dataset()
true_labels = data_loader.get_classification().labels
num_features = data_loader.data_attributes
feature_names_list = feature_names[dataset_name]

# Định nghĩa màu sắc cho các lớp
colors = ['blue', 'yellow', 'red'] # Setosa: xanh dương, Versicolor:
vàng, Virginica: đỏ
labels = ['Setosa', 'Versicolor', 'Virginica']

# Tạo figure với 3 subplot cho Iris
if dataset_name == "Iris":
    fig, axes = plt.subplots(1, 3, figsize=(15, 5))
    axes = axes.flatten()
    feature_pairs = [(0, 1), (0, 2), (0, 3)] # sepal_length vs
(sepal_width, petal_length, petal_width)
else:
    fig, axes = plt.subplots(1, 3, figsize=(15, 5))
    axes = axes.flatten()
    feature_pairs = [(0, 1), (0, 2), (0, 3)]

# Định nghĩa phạm vi trục dựa trên hình ảnh đã cung cấp
axis_limits = {
    0: (4, 8),      # sepal_length: 4 đến 8
    1: (2, 4.5),    # sepal_width: 2 đến 4.5
    2: (1, 7),      # petal_length: 1 đến 7
    3: (0, 2.5)     # petal_width: 0 đến 2.5
}

# Vẽ từng subplot
for idx, (feat_x, feat_y) in enumerate(feature_pairs):
    ax = axes[idx]
    x_label = feature_names_list[feat_x].lower().replace(' ', '_')
    y_label = feature_names_list[feat_y].lower().replace(' ', '_')
    ax.set_xlabel(f"{x_label}")
    ax.set_ylabel(f"{y_label}")

    # Đặt phạm vi trục
    ax.set_xlim(axis_limits[feat_x])
    ax.set_ylim(axis_limits[feat_y])

    # Vẽ từng điểm trung tâm (sử dụng giá trị gốc)

```

```

        for i in range(dataset.get_size()):
            entry = dataset.get_entry(i)
            label = true_labels[i]
            values = entry.get_original_values() # Sử dụng giá trị gốc

            x_val = values[feat_x]
            y_val = values[feat_y]
            color = colors[label]

            # Vẽ điểm trung tâm
            ax.scatter(x_val, y_val, c=color, s=100, alpha=0.8)

        # Thêm chú thích (legend) vào subplot đầu tiên
        handles = [plt.Line2D([0], [0], marker='o', color='w',
markerfacecolor=colors[i], markersize=10, label=labels[i]) for i in
range(3)]
        axes[0].legend(handles=handles, title="Classes")

        # Điều chỉnh layout
        plt.tight_layout()
        plt.suptitle(f"{dataset_name} Visualization (PDF={pdf_name})",
y=1.05)
        plt.show()

```

12. Luồng thực thi chính

```

# --- Main Execution Logic ---
def run_uk_medoids(dataset_name, k, pdf_name):
    try:
        data_loader = DataLoader(datasets[dataset_name],
true_labels[dataset_name], original_data[dataset_name])
        data_loader.generate_uncertainty(pdf_name,
dataset_name=dataset_name)
        dataset = data_loader.get_dataset()

        file_path = f"distances_{dataset_name}_{pdf_name}.txt"
        dpm = DistanceMatrixFile(file_path)
        uk_medoids = UKMedoids(dataset)

        uk_medoids.compute_multivariate_distances(dpm)
        dpm.close()

```

```

        init_strategy = 'random'
        predicted_labels = uk_medoids.execute(file_path, k,
init_method=init_strategy)

        if predicted_labels is None or len(predicted_labels) !=
dataset.get_size(): return np.nan, None

        actual_true_labels = data_loader.get_classification().labels
        evaluation = ClusteringExternalEvaluation(actual_true_labels,
predicted_labels)
        fmeasure = evaluation.f_measure()
        return fmeasure, data_loader
    except Exception as e:
        print(f"\n!!! ERROR in run_uk_medoids ({dataset_name},
{pdf_name}): {e}")
        return np.nan, None

def run_k_medoids_standard(dataset_name, k, pdf_name):
    try:
        data_loader = DataLoader(datasets[dataset_name],
true_labels[dataset_name], original_data[dataset_name])
        dataset = data_loader.get_dataset()

        data_samples = np.array([dataset.get_entry(i).get_values() for i
in range(dataset.get_size())])
        n_samples = len(data_samples)

        initial_medoids = np.random.choice(n_samples, k, replace=False)
        medoids = initial_medoids.copy()
        labels = np.zeros(n_samples, dtype=int)
        max_iter = 100

        for iteration in range(max_iter):
            old_labels = labels.copy()
            for i in range(n_samples):
                distances = [np.sum((data_samples[i] -
data_samples[medoid])**2) for medoid in medoids]
                labels[i] = np.argmin(distances)

            if np.array_equal(labels, old_labels):
                break

```

```

        for cluster_idx in range(k):
            cluster_points_idx = np.where(labels == cluster_idx)[0]
            if len(cluster_points_idx) == 0:
                continue

            min_cost = float('inf')
            best_medoid = medoids[cluster_idx]
            for candidate_idx in cluster_points_idx:
                cost = 0
                for point_idx in cluster_points_idx:
                    if point_idx != candidate_idx:
                        cost += np.sum((data_samples[candidate_idx] -
data_samples[point_idx])**2)
                if cost < min_cost:
                    min_cost = cost
                    best_medoid = candidate_idx
            medoids[cluster_idx] = best_medoid

    actual_true_labels = data_loader.get_classification().labels
    evaluation = ClusteringExternalEvaluation(actual_true_labels,
labels)

    fmeasure = evaluation.f_measure()
    return fmeasure

except Exception as e:
    print(f"\n!!! ERROR in run_k_medoids_standard ({dataset_name},
{pdf_name}): {e}")
    return np.nan

def print_fmeasure_table_with_comparison():
    pdf_types = ["uniformMV", "normal", "binomial"]
    all_datasets = ["Iris", "Wine"]
    num_runs = 100

    aggregated_results_uk = {ds_name: {pdf_name: [] for pdf_name in
pdf_types} for ds_name in all_datasets}
    aggregated_results_std = {ds_name: {pdf_name: [] for pdf_name in
pdf_types} for ds_name in all_datasets}
    data_loaders = {ds_name: {pdf_name: None for pdf_name in pdf_types}
for ds_name in all_datasets}

    total_start_time = time.time()

```

```

for run_num in range(num_runs):
    for dataset_name in all_datasets:
        for pdf_name in pdf_types:
            k = k_values[dataset_name]

            # Chạy UK-Medoids và lưu data_loader để trực quan hóa
            fmeasure_uk, data_loader = run_uk_medoids(dataset_name,
k, pdf_name)

            if not np.isnan(fmeasure_uk):
                aggregated_results_uk[dataset_name][pdf_name].append(
fmeasure_uk)

            if data_loader is not None:
                data_loaders[dataset_name][pdf_name] = data_loader

            # Chạy K-Medoids chuẩn
            fmeasure_std = run_k_medoids_standard(dataset_name, k,
pdf_name)

            if not np.isnan(fmeasure_std):
                aggregated_results_std[dataset_name][pdf_name].append
(fmeasure_std)

            averaged_results_uk = {ds_name: {pdf: np.nanmean(f_measures) if
f_measures else np.nan
                                for pdf, f_measures in
pdf_res.items()}}

                                for ds_name, pdf_res in
aggregated_results_uk.items()}}
            averaged_results_std = {ds_name: {pdf: np.nanmean(f_measures) if
f_measures else np.nan
                                for pdf, f_measures in
pdf_res.items()}}

                                for ds_name, pdf_res in
aggregated_results_std.items()}}

            print("\n=== F-Measure Comparison: UK-Medoids vs K-Medoids ===")
            header = "| Dataset | PDF Type | UK-Medoids | K-Medoids |"
            separator = "+-----+-----+-----+-----+-----+
-+"

            print(separator)
            print(header)
            print(separator)

```



```

for i, dataset_name in enumerate(all_datasets):
    if i > 0:
        print(separator.replace("-", "-"))
    for j, pdf_name in enumerate(pdf_types):
        avg_fmeasure_uk =
averaged_results_uk[dataset_name].get(pdf_name, np.nan)
        avg_fmeasure_std =
averaged_results_std[dataset_name].get(pdf_name, np.nan)

        fmeasure_uk_str = f"{avg_fmeasure_uk:.2f}" if not
np.isnan(avg_fmeasure_uk) else "NaN"
        fmeasure_std_str = f"{avg_fmeasure_std:.2f}" if not
np.isnan(avg_fmeasure_std) else "NaN"

        pdf_display = pdf_name.replace("uniformMV",
"Uniform").capitalize()
        dataset_disp = dataset_name if j == 0 else ""
        print(f"| {dataset_disp:<7} | {pdf_display:<9} |
{fmeasure_uk_str:>10} | {fmeasure_std_str:>19} |")
    print(separator)

# Trực quan hóa dữ liệu Iris cho từng loại PDF
for pdf_name in pdf_types:
    dataset_name = "Iris"
    data_loader = data_loaders[dataset_name][pdf_name]
    if data_loader is not None:
        visualize_uncertainty(dataset_name, pdf_name, data_loader)

if __name__ == "__main__":
    print_fmeasure_table_with_comparison()

```

3.1.3. Đánh giá

Để đánh giá hiệu quả của thuật toán UK-medoids, code sử dụng chỉ số **F-Measure**, một chỉ số phổ biến để đánh giá độ chính xác của thuật toán phân lớp không giám sát. F-Measure được tính dựa trên nhãn thật và nhãn dự đoán, với công thức:

$$F\text{-Measure} = 2 \cdot \frac{precision \cdot recall}{precision + recall}$$

Trong đó:

- $precision = \frac{TP}{TP + FP}$

- $\text{recall} = \frac{TP}{TP+FN}$
- TP (True Positive): Số cặp mẫu cùng lớp và được dự đoán cùng cụm.
- FP (False Positive): Số cặp mẫu khác lớp nhưng được dự đoán cùng cụm.
- FN (False Negative): Số cặp mẫu cùng lớp nhưng được dự đoán khác cụm.

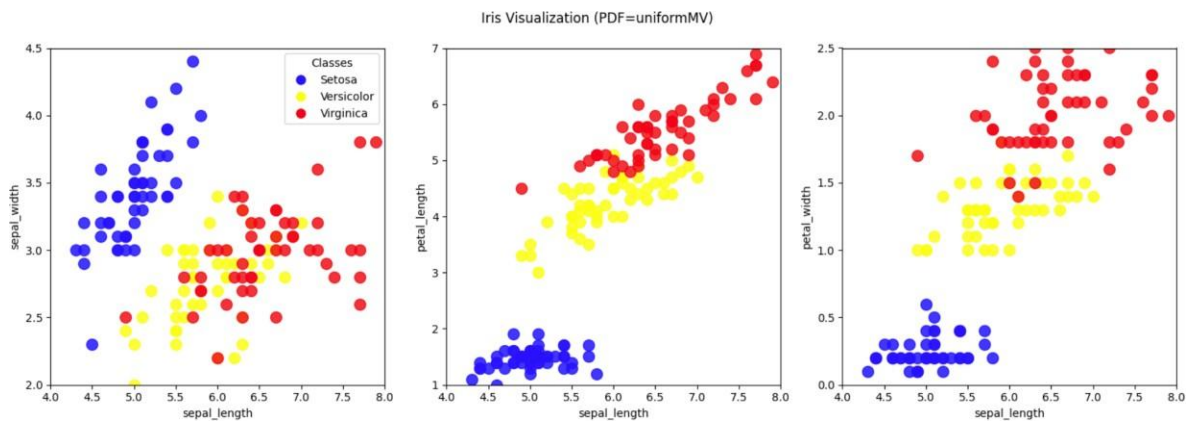
Dataset	PDF Type	UK-medoids	K-medoids
Iris	Uniform	0.73	0.72
	Normal	0.75	0.73
	Binomial	0.74	0.72
Wine	Uniform	0.77	0.75
	Normal	0.72	0.73
	Binomial	0.71	0.75

Bảng 3.2. Bảng so sánh F-Measure giữa UK-Medoids và K-Medoids

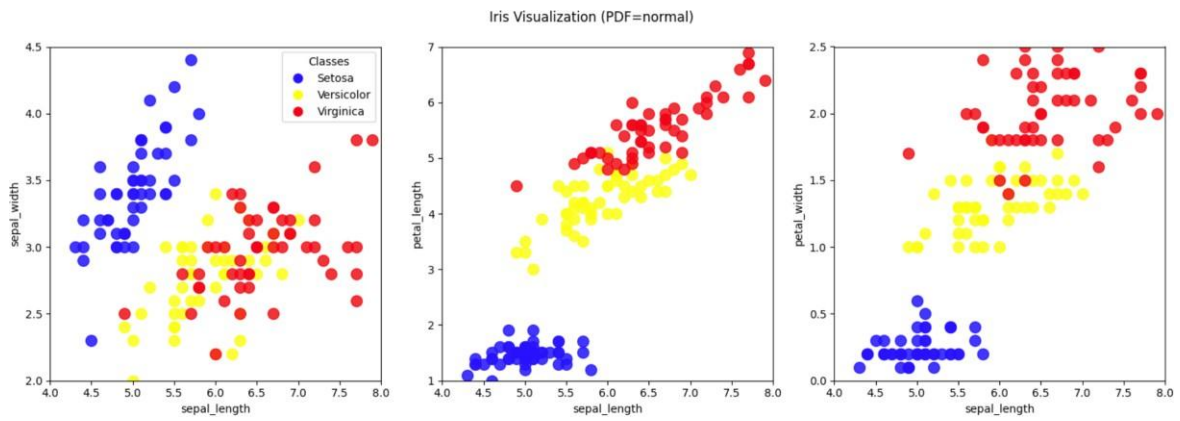
Phân tích kết quả

- Tất cả các giá trị đều nằm trong khoảng 0.71-0.77, cho thấy cả hai thuật toán đều hoạt động khá tốt trên các bộ dữ liệu này (F-measure càng gần 1 càng tốt).
- Sự chênh lệch không lớn: Sự khác biệt về hiệu suất giữa hai thuật toán khá nhỏ (dao động từ 0.01 đến 0.03), cho thấy không có thuật toán nào vượt trội hoàn toàn so với thuật toán còn lại.

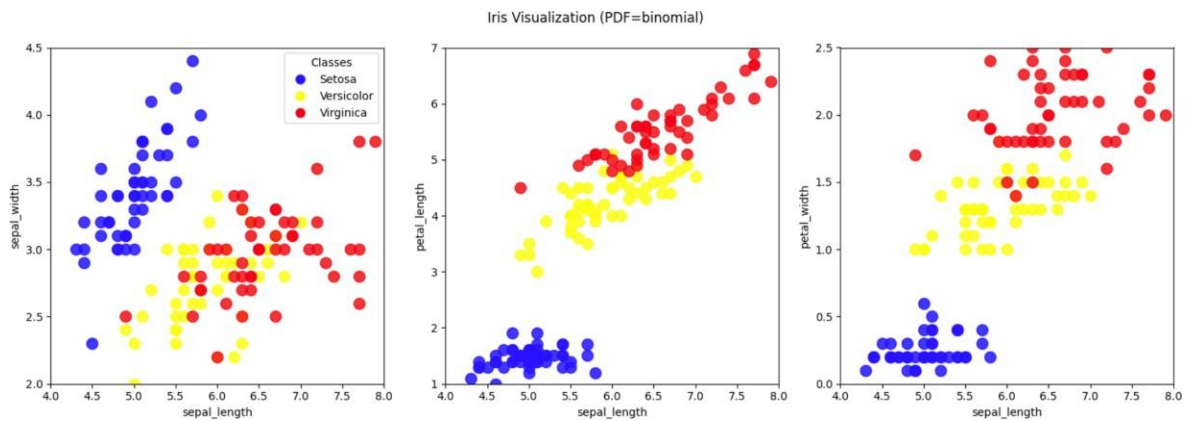
Trực quan hóa kết quả



Hình 3.1. Minh họa phân lớp tập Iris với phân phối xác suất Uniform



Hình 3.2. Minh họa phân lớp tập Iris với phân phối xác suất Normal



Hình 3.3. Minh họa phân lớp tập Iris với phân phối xác suất Binomial

3.2. Cài đặt thuật toán UK-means

3.2.1. Bộ dữ liệu

○ Nguồn dữ liệu:

Dữ liệu được lấy từ kho lưu trữ UCI Machine Learning Repository, thông qua gói ucimlrepo trong Python [6]. Cụ thể, sử dụng 2 tập dữ liệu phổ biến:

- Iris: Tập dữ liệu về hoa Iris [9]
- Wine: Tập dữ liệu về rượu vang [10]

Các tập dữ liệu này được tải trực tiếp bằng hàm fetch_ucirepo từ gói ucimlrepo, không cần tải file CSV từ web hay đọc từ nguồn cục bộ.

○ Đọc dữ liệu:

- Sử dụng gói ucimlrepo để tải các tập dữ liệu Iris (ID=53), Wine (ID=109).
- Các tập dữ liệu được lưu vào biến iris và wine.

○ Tiền xử lý dữ liệu:

1. Lọc và chuẩn hóa nhãn:

- Với tập Iris, nhãn gốc là chuỗi nên cần ánh xạ thành số nguyên:
 - Iris: {'Iris-setosa': 0, 'Iris-versicolor': 1, 'Iris-virginica': 2}
- Với tập Wine, nhãn đã là số nguyên, nhưng code kiểm tra để đảm bảo tính hợp lệ.

2. Chuẩn hóa đặc trưng:

- Sử dụng StandardScaler từ gói sklearn.preprocessing để chuẩn hóa dữ liệu, đảm bảo các thuộc tính có trung bình bằng 0 và độ lệch chuẩn bằng 1.

3. Chuyển đổi thành dữ liệu không chắc chắn:

- Mỗi mẫu dữ liệu ban đầu được chuyển thành dữ liệu không chắc chắn (uncertain data) bằng cách sử dụng các phân phối xác suất:
 - UniformMV (phân phối đều đa biến)
 - Normal (phân phối chuẩn đa biến)
 - Binomial (phân phối đều đơn biến)
- Tỷ lệ mở rộng đoạn (percentage) được điều chỉnh riêng cho từng tập dữ liệu và từng loại phân phối.

Sau khi tiền xử lý, các tập dữ liệu có đặc điểm như sau:

Tập dữ liệu	Số mẫu	Số thuộc tính	Số lớp	Mô tả nhãn/lớp
Iris	150	4	3	0 (setosa), 1 (versicolor), 2 (virginica)
Wine	178	13	3	1, 2, 3 (ba loại rượu vang)

Bảng 3.3. Mô tả dữ liệu đầu vào cho thuật toán UK-means sau tiền xử lý

- **Dạng dữ liệu:** Sau khi chuyển đổi, mỗi thuộc tính của mỗi mẫu không phải là một giá trị cố định, mà được biểu diễn bằng một phân phối xác suất.
- **Nhãn/lớp:** Mặc dù các tập dữ liệu có nhãn, thuật toán UK-means là một thuật toán không giám sát, nên nhãn chỉ được sử dụng để đánh giá kết quả phân cụm.

3.2.2. Xây dựng mô hình

Các bước từ 1 đến 11 được thực hiện tương tự như trong phần UK-Medoids đã trình bày trước đó. Tại phần này, chúng tôi bổ sung thuật toán UK-means nhằm mở rộng so sánh và đánh giá hiệu quả giữa các phương pháp.

1. Import các thư viện hỗ trợ
2. Cấu hình ban đầu
3. Tải và tiền xử lý dữ liệu
4. Các lớp cốt lõi
5. Các lớp phân phân xác suất
6. Tính khoảng cách không chắc chắn
7. Tải dữ liệu với TUNED Uncertainty Generation
8. Xử lý tệp ma trận khoảng cách
9. Thuật toán UK-medoids
10. Đánh giá kết quả
11. Trực quan hóa dữ liệu
12. Thuật toán UK-means

```
# --- UK-Means Algorithm ---
class UKMeans:
    def __init__(self, dataset):
        self.d = dataset
        self.dist = None
        self.n = dataset.get_size()
        self.dim = len(dataset.get_entry(0).get_values()) if
dataset.get_size() > 0 else 0

    def compute_multivariate_distances(self, dpm):
        calc = AnalyticalDistanceCalculator(self.d)
        self.dist = calc.compute_distance_matrix(dpm)

    def load_distances(self, fp):
        dpm = DistanceMatrixFile(fp)
        self.dist = dpm.load_matrix(self.n)

    def _init_random(self, k):
        # Chọn k điểm ngẫu nhiên làm centroids ban đầu
        indices = np.random.choice(self.n, k, replace=False)
        return np.array([self.d.get_entry(i).get_values() for i in
indices])

    def expected_distance(self, entry, centroid):
```

```

        # Tính kỳ vọng khoảng cách giữa đối tượng không chắc chắn và centroid
        if entry.u_values is None:
            return np.sum((entry.get_values() - centroid) ** 2)

        if isinstance(entry.u_values, MultivariateGaussian):
            return np.sum((entry.u_values.mu - centroid) ** 2) +
np.trace(entry.u_values.sigma)
        elif isinstance(entry.u_values, MultivariateUniform):
            ex = (entry.u_values.min_values + entry.u_values.max_values)
/ 2.0

            r = entry.u_values.max_values - entry.u_values.min_values
            vx = np.where(r > 1e-12, (r**2) / 12.0, 0)
            return np.sum((ex - centroid) ** 2 + vx)
        elif isinstance(entry.u_values, list):
            total_sq_dist = 0
            for d, p in enumerate(entry.u_values):
                yd = centroid[d]
                if isinstance(p, Binomial):
                    a, b = p.lower_bound, p.upper_bound
                    r = b - a
                    ex = (a + b) / 2.0
                    vx = (r**2) / 12.0 if r > 1e-12 else 0
                    total_sq_dist += (ex - yd) ** 2 + vx
                else:
                    # Monte Carlo cho PDF phức tạp
                    samples = [p.get_random_sample() for _ in
range(1000)]

                    total_sq_dist += np.mean([(x - yd) ** 2 for x in
samples])
            return total_sq_dist
        else:
            # Monte Carlo cho các PDF phức tạp
            samples = [entry.get_random_sample() for _ in range(1000)]
            return np.mean([np.sum((s - centroid) ** 2) for s in
samples])

def k_means(self, k, max_iter=100, init_method='random'):
    if k > self.n:
        raise ValueError(f"k={k} > n={self.n}")
    if k == 0:
        return np.array([]), np.array([])

    # Khởi tạo centroids từ k điểm dữ liệu ngẫu nhiên
    indices = np.random.choice(self.n, k, replace=False)
    centroids = np.array([self.d.get_entry(i).get_values() for i in
indices])

    labels = -np.ones(self.n, dtype=int)
    best_cost = float('inf')

```

```

best_centroids = np.copy(centroids)
best_labels = np.copy(labels)

for it in range(max_iter):
    # Bước gán: Gán mỗi điểm vào cụm gần nhất
    new_labels = np.zeros(self.n, dtype=int)
    current_cost = 0

    for i in range(self.n):
        entry = self.d.get_entry(i)
        min_dist = float('inf')
        min_idx = -1

        for j in range(k):
            dist = self._expected_distance(entry, centroids[j])
            if dist < min_dist:
                min_dist = dist
                min_idx = j

        new_labels[i] = min_idx
        current_cost += min_dist

    # Kiểm tra hội tụ
    cost_improved = current_cost < best_cost - 1e-9
    assignments_stable = np.array_equal(new_labels, labels)

    if not cost_improved and assignments_stable and it > 0:
        break

    if cost_improved:
        best_cost = current_cost
        best_centroids = np.copy(centroids)
        best_labels = np.copy(new_labels)

    labels = new_labels

    # Bước cập nhật: Tính lại centroids
    new_centroids = np.zeros((k, self.dim))
    cluster_sizes = np.zeros(k)

    for j in range(k):
        cluster_indices = np.where(labels == j)[0]
        if len(cluster_indices) == 0:
            continue # Giữ nguyên centroid cũ nếu cụm trống

        # Tính trung bình của cụm theo công thức UK-means
        for idx in cluster_indices:
            entry = self.d.get_entry(idx)
            if entry.u_values is None:
                new_centroids[j] += entry.get_values()

```

```

        elif isinstance(entry.u_values,
MultivariateGaussian):
            new_centroids[j] += entry.u_values.mu
        elif isinstance(entry.u_values, MultivariateUniform):
            new_centroids[j] += (entry.u_values.min_values +
entry.u_values.max_values) / 2.0
        elif isinstance(entry.u_values, list):
            feature_means = []
            for d, pdf in enumerate(entry.u_values):
                if isinstance(pdf, Binomial):
                    feature_means.append((pdf.lower_bound +
pdf.upper_bound) / 2.0)
                else:
                    samples = [pdf.get_random_sample() for _
in range(1000)]
                    feature_means.append(np.mean(samples))
            new_centroids[j] += np.array(feature_means)
        else:
            samples = [entry.get_random_sample() for _ in
range(1000)]
            new_centroids[j] += np.mean(samples, axis=0)

        new_centroids[j] /= len(cluster_indices)
        cluster_sizes[j] = len(cluster_indices)

        # Nếu các centroids không thay đổi, dùng
        if np.all(np.abs(centroids - new_centroids) < 1e-9) and it >
0:
            break

        centroids = new_centroids

    return best_labels, best_centroids

def execute(self, fp, k, init_method='random'):
    labels, centroids = self.k_means(k, init_method=init_method)
    return labels

```

13. Luồng thực thi chính

```

# --- Main Execution Logic ---
def run_uk_medoids(dataset_name, k, pdf_name):
    try:
        data_loader = DataLoader(datasets[dataset_name],
true_labels[dataset_name], original_data[dataset_name])
        data_loader.generate_uncertainty(pdf_name,
dataset_name=dataset_name)
        dataset = data_loader.get_dataset()

        file_path = f"distances {dataset name} {pdf name}.txt"

```



```

dpm = DistanceMatrixFile(file_path)
uk_medoids = UKMedoids(dataset)

uk_medoids.compute_multivariate_distances(dpm)
dpm.close()

init_strategy = 'random'
predicted_labels = uk_medoids.execute(file_path, k,
init_method=init_strategy)

if predicted_labels is None or len(predicted_labels) !=
dataset.get_size(): return np.nan, None

actual_true_labels = data_loader.get_classification().labels
evaluation = ClusteringExternalEvaluation(actual_true_labels,
predicted_labels)
fmeasure = evaluation.f_measure()
return fmeasure, data_loader
except Exception as e:
    print(f"\n!!! ERROR in run_uk_medoids ({dataset_name},
{pdf_name}): {e}")
    return np.nan, None

def run_k_medoids_standard(dataset_name, k, pdf_name):
    try:
        data_loader = DataLoader(datasets[dataset_name],
true_labels[dataset_name], original_data[dataset_name])
        dataset = data_loader.get_dataset()

        data_samples = np.array([dataset.get_entry(i).get_values() for i
in range(dataset.get_size())])
        n_samples = len(data_samples)

        initial_medoids = np.random.choice(n_samples, k, replace=False)
        medoids = initial_medoids.copy()
        labels = np.zeros(n_samples, dtype=int)
        max_iter = 100

        for iteration in range(max_iter):
            old_labels = labels.copy()
            for i in range(n_samples):
                distances = [np.sum((data_samples[i] -
data_samples[medoid])**2) for medoid in medoids]
                labels[i] = np.argmin(distances)

            if np.array_equal(labels, old_labels):
                break

            for cluster_idx in range(k):
                cluster_points_idx = np.where(labels == cluster_idx)[0]

```

```

        if len(cluster_points_idx) == 0:
            continue
        min_cost = float('inf')
        best_medoid = medoids[cluster_idx]
        for candidate_idx in cluster_points_idx:
            cost = 0
            for point_idx in cluster_points_idx:
                if point_idx != candidate_idx:
                    cost += np.sum((data_samples[candidate_idx] -
data_samples[point_idx])**2)
            if cost < min_cost:
                min_cost = cost
                best_medoid = candidate_idx
            medoids[cluster_idx] = best_medoid

        actual_true_labels = data_loader.get_classification().labels
        evaluation = ClusteringExternalEvaluation(actual_true_labels,
labels)

        fmeasure = evaluation.f_measure()
        return fmeasure
    except Exception as e:
        print(f"\n!!! ERROR in run_k_medoids_standard ({dataset_name},
{pdf_name}): {e}")
        return np.nan
# Function to run UK-means
def run_uk_means(dataset_name, k, pdf_name):
    try:
        data_loader = DataLoader(datasets[dataset_name],
true_labels[dataset_name], original_data[dataset_name])
        data_loader.generate_uncertainty(pdf_name,
dataset_name=dataset_name)
        dataset = data_loader.get_dataset()

        uk_means = UKMeans(dataset)
        predicted_labels, _ = uk_means.k_means(k)

        if predicted_labels is None or len(predicted_labels) !=
dataset.get_size():
            return np.nan, None

        actual_true_labels = data_loader.get_classification().labels
        evaluation = ClusteringExternalEvaluation(actual_true_labels,
predicted_labels)
        fmeasure = evaluation.f_measure()
        return fmeasure, data_loader
    except Exception as e:
        print(f"\n!!! ERROR in run_uk_means ({dataset_name}, {pdf_name}):
{e}")
        traceback.print_exc()
        return np.nan, None

```

```

def print_fmeasure_table_with_comparison():
    pdf_types = ["uniformMV", "normal", "binomial"]
    all_datasets = ["Iris", "Wine"]
    num_runs = 50 # Giảm số lượng chạy để tiết kiệm thời gian

    aggregated_results_uk_medoids = {ds_name: {pdf_name: [] for pdf_name
in pdf_types} for ds_name in all_datasets}
    aggregated_results_uk_means = {ds_name: {pdf_name: [] for pdf_name in
pdf_types} for ds_name in all_datasets}
    aggregated_results_std = {ds_name: {pdf_name: [] for pdf_name in
pdf_types} for ds_name in all_datasets}
    data_loaders = {ds_name: {pdf_name: None for pdf_name in pdf_types}
for ds_name in all_datasets}

    total_start_time = time.time()

    for run_num in range(num_runs):
        print(f"Lần chạy {run_num+1}/{num_runs}")
        for dataset_name in all_datasets:
            for pdf_name in pdf_types:
                k = k_values[dataset_name]

                # Chạy UK-Medoids
                fmeasure_uk_medoids, data_loader =
run_uk_medoids(dataset_name, k, pdf_name)
                if not np.isnan(fmeasure_uk_medoids):
                    aggregated_results_uk_medoids[dataset_name][pdf_name]
.append(fmeasure_uk_medoids)
                if data_loader is not None:
                    data_loaders[dataset_name][pdf_name] = data_loader

                # Chạy UK-Means
                fmeasure_uk_means, _ = run_uk_means(dataset_name, k,
pdf_name)
                if not np.isnan(fmeasure_uk_means):
                    aggregated_results_uk_means[dataset_name][pdf_name].a
ppend(fmeasure_uk_means)

                # Chạy K-Medoids chuẩn
                fmeasure_std = run_k_medoids_standard(dataset_name, k,
pdf_name)
                if not np.isnan(fmeasure_std):
                    aggregated_results_std[dataset_name][pdf_name].append
(fmeasure_std)

            averaged_results_uk_medoids = {ds_name: {pdf: np.nanmean(f_measures)
if f_measures else np.nan
                                for pdf, f_measures in
pdf_res.items()}}

```

```

        for ds_name, pdf_res in
aggregated_results_uk_medoids.items()
    averaged_results_uk_means = {ds_name: {pdf: np.nanmean(f_measures) if
f_measures else np.nan
                                for pdf, f_measures in
pdf_res.items()}}
        for ds_name, pdf_res in
aggregated_results_uk_means.items()
    averaged_results_std = {ds_name: {pdf: np.nanmean(f_measures) if
f_measures else np.nan
                                for pdf, f_measures in
pdf_res.items()}}
        for ds_name, pdf_res in
aggregated_results_std.items()

    print("\n=== F-Measure Comparison: UK-Medoids vs UK-Means vs K-
Medoids ===")
    header = "| Dataset | PDF Type | UK-Medoids | UK-Means | K-
Medoids |"
    separator = "+-----+-----+-----+-----+-----+
-----+"

    print(separator)
    print(header)
    print(separator)

    for i, dataset_name in enumerate(all_datasets):
        if i > 0:
            print(separator.replace("-", "-"))
            for j, pdf_name in enumerate(pdf_types):
                avg_fmeasure_uk_medoids =
averaged_results_uk_medoids[dataset_name].get(pdf_name, np.nan)
                avg_fmeasure_uk_means =
averaged_results_uk_means[dataset_name].get(pdf_name, np.nan)
                avg_fmeasure_std =
averaged_results_std[dataset_name].get(pdf_name, np.nan)

                fmeasure_uk_medoids_str = f"{avg_fmeasure_uk_medoids:.2f}" if
not np.isnan(avg_fmeasure_uk_medoids) else "NaN"
                fmeasure_uk_means_str = f"{avg_fmeasure_uk_means:.2f}" if not
np.isnan(avg_fmeasure_uk_means) else "NaN"
                fmeasure_std_str = f"{avg_fmeasure_std:.2f}" if not
np.isnan(avg_fmeasure_std) else "NaN"

                pdf_display = pdf_name.replace("uniformMV",
"Uniform").capitalize()
                dataset_disp = dataset_name if j == 0 else ""
                print(f"| {dataset_disp:<7} | {pdf_display:<9} |
{fmeasure_uk_medoids_str:>10} | {fmeasure_uk_means_str:>10} |
{fmeasure_std_str:>10} |")

```

```

print(separator)

# Trực quan hóa dữ liệu Iris cho từng loại PDF
for pdf_name in pdf_types:
    dataset_name = "Iris"
    data_loader = data_loaders[dataset_name][pdf_name]
    if data_loader is not None:
        visualize_uncertainty(dataset_name, pdf_name, data_loader)

if __name__ == "__main__":
    print_fmeasure_table_with_comparison()

```

3.3.3. Đánh giá

Để đánh giá hiệu quả của thuật toán UK-means, chúng tôi sử dụng chỉ số F-Measure, được tính dựa trên:

$$F\text{-Measure} = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$

Trong đó:

- $\text{precision} = \frac{TP}{TP+FP}$
- $\text{recall} = \frac{TP}{TP+FN}$

Với:

- TP (True Positive): Số cặp mẫu cùng lớp và được dự đoán cùng cụm.
- FP (False Positive): Số cặp mẫu khác lớp nhưng được dự đoán cùng cụm.
- FN (False Negative): Số cặp mẫu cùng lớp nhưng được dự đoán khác cụm.
- TN (True Negative): Số cặp mẫu khác lớp và được dự đoán khác cụm.

Chỉ số F-Measure cao hơn (gần 1) chỉ ra rằng thuật toán phân cụm phù hợp hơn với cấu trúc lớp thật sự của dữ liệu.

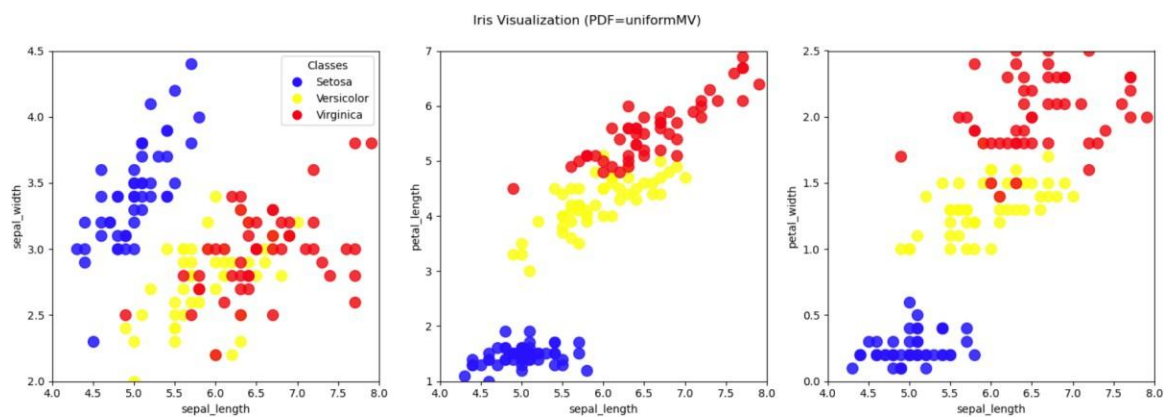
Dataset	PDF Type	UK-medoids	UK-means	K-medoids
Iris	Uniform	0.74	0.73	0.72
	Normal	0.76	0.73	0.73
	Binomial	0.73	0.72	0.74
Wine	Uniform	0.74	0.89	0.75
	Normal	0.72	0.91	0.73
	Binomial	0.71	0.92	0.73

Bảng 3.4. Bảng so sánh F-Measure giữa UK-means, UK-Medoids và K-Medoids

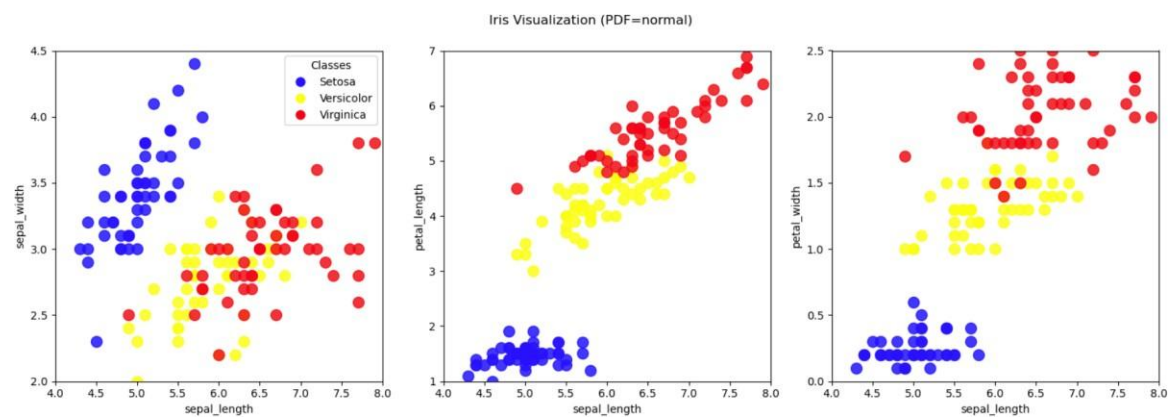
Phân tích kết quả

- UK-means tỏ ra hiệu quả hơn đáng kể so với UK-medoids và K-medoids trên tập dữ liệu Wine, đặc biệt khi kết hợp với phân phối Normal và Binomial.
- UK-medoids phù hợp hơn với Iris khi dùng phân phối Normal.
- Sự khác biệt giữa các thuật toán thể hiện rõ hơn ở tập dữ liệu phức tạp hơn như Wine, phản ánh khả năng thích nghi của từng thuật toán với đặc trưng dữ liệu và phân phối xác suất được sử dụng.

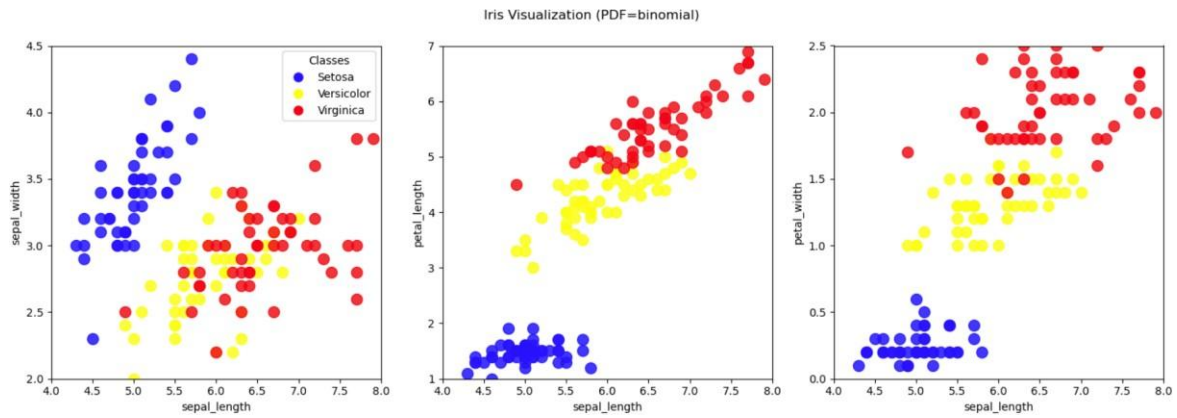
Trực quan hóa dữ liệu



Hình 3.4. Minh họa phân lớp tập Iris với phân phối xác suất Uniform



Hình 3.5. Minh họa phân lớp tập Iris với phân phối xác suất Normal



Hình 3.6. Minh họa phân lớp tập Iris với phân phối xác suất Binomial

3.3. So sánh, đánh giá các thuật toán

[1]ELKI là một phần mềm khai phá dữ liệu mã nguồn mở (AGPLv3) được viết bằng Java, hướng đến người dùng trong nghiên cứu và phát triển thuật toán, đặc biệt nhấn mạnh vào các phương pháp không giám sát như phân lớp và phát hiện điểm bất thường.[2] Đây là phần mềm đã được ứng dụng trong học tập ở trường đại học LMU Munich ([3]trường đại học được xếp hạng thứ 38 thế giới về CNTT năm 2024 theo đánh giá của world university ranking)

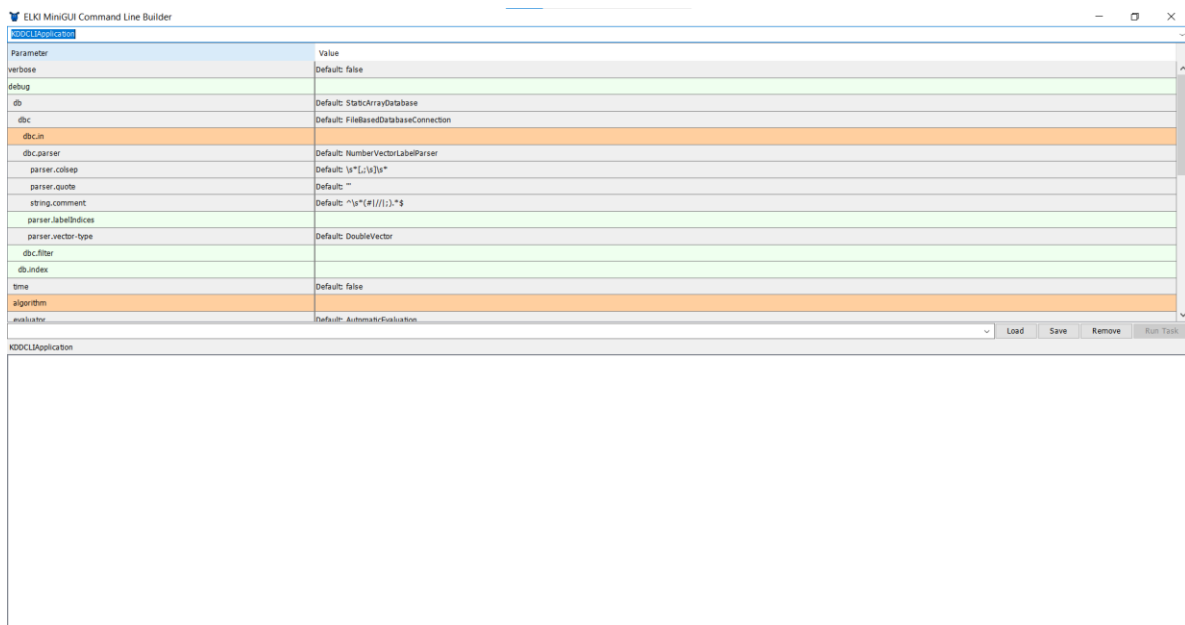
❖ Hướng dẫn cài đặt và sử dụng ELKI [4]

Để sử dụng phần mềm ELKI (Environment for Developing KDD-Applications Supported by Index-Structures), trước tiên người dùng cần tải về gói cài đặt tại địa chỉ: <https://elki-project.github.io/releases/release0.8.0/elki-0.8.0.tar.gz>

Sau khi giải nén và khởi chạy tệp **elki.bat**, giao diện người dùng chính của ELKI sẽ hiển thị.

1. Giao diện lựa chọn tác vụ

Tại dòng đầu tiên của giao diện, người dùng có thể lựa chọn tác vụ sẽ thực thi. Mặc định, ELKI khởi tạo với tác vụ **KDDCLIApplication**, tức là phần mềm đang được thiết lập để xây dựng các tham số phục vụ cho quá trình khai phá dữ liệu (KDD - Knowledge Discovery in Databases).



2. Bảng thiết lập tham số

Phía dưới khu vực tác vụ là bảng thiết lập các tham số. ELKI cung cấp một số lượng lớn tham số có thể cấu hình. Các tham số này được hiển thị với mã màu nhằm thể hiện mức độ quan trọng và tính bắt buộc của từng mục:

- **Màu cam:** Là các tham số bắt buộc, bắt buộc phải được điền đầy đủ để tiến trình chạy thành công.
- **Màu xanh:** Là các tham số tùy chọn, người dùng có thể điền hoặc không. Tuy nhiên, trong một số trường hợp cụ thể, việc không điền tham số này có thể gây lỗi trong quá trình thực thi.
- **Màu xám:** Là các tham số mặc định, có thể để trống mà không ảnh hưởng đến quá trình thực thi.

3. Trình quản lý cấu hình

Ngay bên dưới bảng tham số là khu vực quản lý cấu hình. Tại đây, người dùng có thể lưu, thêm hoặc xóa các thiết lập tham số đã cấu hình trước đó, giúp dễ dàng tái sử dụng hoặc chỉnh sửa cho các lần chạy sau.

4. Bảng hiển thị thông báo hệ thống

Cuối cùng, khu vực dưới cùng của giao diện là bảng trắng dùng để hiển thị các thông báo từ hệ thống, bao gồm thông tin quá trình thực thi, cảnh báo hoặc lỗi trong quá trình sử dụng phần mềm.

5. Cài đặt UK-means và CK-means trên ELKI

- Nguồn dữ liệu:

Dữ liệu được lấy từ kho lưu trữ UCI Machine Learning Repository, thông qua định dạng CSV(chuyển từ dạng data sang CSV). Cụ thể, chúng tôi sử dụng 2 tập dữ liệu phổ biến:

- Iris: Tập dữ liệu về hoa Iris [9]
- Wine: Tập dữ liệu về rượu vang [10]

- Các thông số chung

dbc.in	E:\User\Downloads\IRIS1.csv
--------	-----------------------------

dbc.in nơi gán đường link dẫn đến chỗ chứa dữ liệu để truyền dữ liệu vào

algorithm	clustering.uncertain.UKMeans
-----------	------------------------------

algorithm	clustering.uncertain.CKMeans
-----------	------------------------------

algorithm là nơi chọn thuật toán, điền **clustering.uncertain.UKMeans** tương ứng với UK-means và **clustering.uncertain.CKMeans** tương ứng với CK-means

kmeans.k	3
----------	---

kmeans.k là nơi điền số cụm, cả tập iris với wine đều có chung **kmeans.k** là 3

dbc.filter	typeconversions.UncertainifyFilter
------------	------------------------------------

[11] **dcb.filter** là nơi chọn bộ lọc, để chuyển đổi dữ liệu thường sang dạng dữ liệu không chắc chắn, bởi vì ở đây chúng tôi đang chọn UncertainfyFilter vì cài đặt thuật toán UK-medoids của chúng tôi đang sử dụng hàm mật độ xác suất liên tục

[12] Vì UK-means không xử lý trực tiếp các phân phối liên tục mà phải xử lý thông qua phân phối rời rạc và cài đặt thuật toán UK-medoids chúng tôi không gán trọng số nên ở đây chúng ta điền UnweightedDiscreteUncertainifier

uo.discrete.generator	UniformUncertainifier
uo.uncertainty.min	0.0
uo.uncertainty.max	10.0

Trong trường hợp cài đặt hàm pdf uniform, ta điền **uo.discrete.generator** là UniformUncertainifier. **uo.uncertainty.min** và **uo.uncertainty.min** lần lượt là cận trên và cận dưới của phân phối uniform.

uo.discrete.generator	SimpleGaussianUncertainifier
uo.uncertainty.min3sigma	0.0
uo.uncertainty.max3sigma	9.390610801544543

Trong trường hợp cài đặt hàm pdf gaussian, ta điền **uo.discrete.generator** là SimpleGaussianUncertainifier. **uo.uncertainty.min3sigma** và **uo.uncertainty.max3sigma** lần lượt là min và max 3-sigma của phân phối gaussian.

uo.quantity.max	500
uo.quantity.min	500

Tham số này quy định số điểm dữ liệu tối đa được tạo ra để mô hình hóa một đối tượng dữ liệu không chắc chắn., [5] nên thiết lập giá trị này ở mức 50, hoặc trong khoảng từ 400 đến 500 để đảm bảo sự cân bằng giữa độ chính xác và hiệu suất tính toán. Ở đây chúng tôi chọn giá trị là 500.

Trong phần triển khai, chúng tôi đang thực hiện lấy kết quả chỉ số F-measure của Pair counting, Pair counting có cùng công thức tính F-measure với thuật toán UK-medoids của chúng tôi.

Dataset	pdf	Thuật toán tự code			ELKI	
		UK-medoids	K-medoids	UK-means	UK-means	CK-means
Iris	Uniform	0.74	0.72	0.73	0.45	0.45
	Normal	0.76	0.73	0.73	0.8318	0.8318
	Binomial	0.73	0.74	0.72	ELKI không hỗ trợ	
Wine	Uniform	0.74	0.75	0.89	0.5904	0.5904
	Normal	0.72	0.73	0.91	0.5904	0.5904
	Binomial	0.71	0.73	0.92	ELKI không hỗ trợ	

Bảng 3.5. Bảng kết quả so sánh các thuật toán và phần mềm ELKI

Đánh giá kết quả

- Hiệu suất của các thuật toán phân cụm có sự khác biệt đáng kể tùy thuộc vào bộ dữ liệu và hàm mật độ được sử dụng.
- Thuật toán UK-means tự cài đặt cho kết quả vượt trội hơn ELKI, đặc biệt trên tập Wine và với các phân phối phi chuẩn như Uniform và Binomial.
- ELKI cho thấy hiệu suất tốt trên bộ dữ liệu Iris với hàm mật độ Normal, nhưng lại có hiệu suất thấp hơn trên bộ dữ liệu Iris và Wine với hàm mật độ Uniform.
- Các thuật toán UK-medoids, K-medoids và UK-means thường cho kết quả ổn định hơn trên các hàm mật độ khác nhau so với ELKI trên bộ dữ liệu Uniform. Tuy nhiên, ELKI lại thể hiện ưu thế với hàm Normal trên bộ Iris.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

KẾT QUẢ:

Trong nghiên cứu này, chúng tôi đã tập trung phân tích và triển khai hai thuật toán phân lớp dữ liệu không chắc chắn là UK-means và UK-medoids. Các thuật toán được áp dụng trên tập dữ liệu mô phỏng với các điểm dữ liệu được biểu diễn dưới dạng phân phối xác suất (Gaussian). Qua quá trình thực nghiệm, cả hai thuật toán đều cho thấy khả năng phân lớp hiệu quả hơn so với các thuật toán truyền thống (K-means, K-medoids) trong môi trường dữ liệu có nhiễu hoặc sai số đo lường.

UK-means hoạt động nhanh và phù hợp với dữ liệu có tính phân bố đồng đều, trong khi UK-medoids tỏ ra ổn định và ít bị ảnh hưởng bởi các điểm ngoại lệ. Kết quả phân lớp phản ánh rõ ràng cấu trúc tiềm ẩn của dữ liệu không chắc chắn, góp phần hỗ trợ các bài toán thực tiễn như định vị GPS, cảm biến môi trường, hoặc phân tích dữ liệu từ thiết bị IoT.

HẠN CHẾ

- Phụ thuộc vào việc chọn số cụm (k).
- Chi phí tính toán Expected Distance cao khi dữ liệu lớn.
- Chỉ áp dụng với phân phối Gaussian đơn giản.
- Mới dừng ở mức mô phỏng, chưa thử nghiệm với dữ liệu thực lớn.
- Kết quả nghiên cứu trên ELKI chưa khớp với các kết quả còn lại
- Các kết quả UK-medoids vẫn còn chưa khớp hoàn toàn với kết quả trong bài báo

HƯỚNG PHÁT TRIỂN

- Nghiên cứu áp dụng độ đo ARI để đánh giá thuật toán
- Thử trên nhiều tập dữ liệu khác (Ecoli, Glass...)
- Nghiên cứu và áp dụng trên bộ dữ liệu tự xây dựng, sử dụng trong phân cụm các bài toán thực tế
- Nghiên cứu thêm các thuật toán khác (UK-means, CK-means, FDBSCAN)

TÀI LIỆU THAM KHẢO

- [1] E. Schubert, A. Koos, T. Emrich, A. Züfle, K. A. Schmid, and A. Zimek, “A Framework for Clustering Uncertain Data,” *Ludwig-Maximilians-Universität München*, Germany
- [2] "Running DBSCAN in ELKI," *Stack Overflow*, May 13, 2014
- [3] "World University Rankings 2024 – LMU Munich," *Times Higher Education*, 2024
- [4] "ELKI: Java Data Mining Toolkit – Tutorial," *ELKI Project*, 2024
- [5] F. Gullo, G. Ponti, and A. Tagarelli, “Clustering Uncertain Data via K-medoids,” *DEIS, University of Calabria*, Italy.
- [6] Dua, D. and Graff, C. (2019). UCI Machine Learning Repository [<http://archive.ics.uci.edu/ml>]. Irvine, CA: University of California, School of Information and Computer Science.
- [7] Pedregosa et al., "Scikit-learn: Machine Learning in Python", *Journal of Machine Learning Research*, 2011.
- [8] Nguyen, H., & Nahavandi, S. (2016). UK-means: A Clustering Algorithm for Uncertain Data. *Knowledge-Based Systems*, 103, 52–64.
- [9] D. Dua and C. Graff, *UCI Machine Learning Repository: Iris Data Set*, University of California, Irvine, School of Information and Computer Sciences.
- [10] D. Dua and C. Graff, *UCI Machine Learning Repository: Wine Data Set*, University of California, Irvine, School of Information and Computer Sciences
- [11] “Answer to ‘How to plot decision boundaries?’,” *Stack Overflow*, Dec. 11, 2016.
- [12] “Answer to ‘How to change the background color of decision regions?’,” *Stack Overflow*, Oct. 13, 2023.